Digital Architecture Framework



Digital Architecture

Framework

DAF v. 7.0

Rev. 16

Oct 2026

Contents

About Digital Architecture Framework (DAF)....3
License....6
Project....7
Installation....7
What's new in DAF 7.0....9
Concepts....11
From Reality to Abstraction....11
From the Model to a View....12
DAF Metamodel....15
Example Model Instance....16
Layers....18
Enterprise Architecture....18
Solution Architecture....18
Design Architecture....19
Domains....19
Business....19
Data....19
Application....19
Technology....19
Extension Domains....20
Viewpoints and Views....21
DAF Concepts Organized in Viewpoints....23
Views and work products....24
Relationships....25
Relationship Strategy....26
Detailed Documentation....27
DAF Concepts Overview....27
DAF Relationship Types....43
DAF Relationships Quick link....52
Visual Characteristics of some DAF stereotypes....59
Appendices....61
Stereotypes and Tagged Values....61
Enumerations....85
Model Checks....89
DAF COLORS....94
RGB Colors....94
HEX Colors....95

1 About Digital Architecture Framework (DAF)

The Digital Architecture Framework (DAF), formerly known as "Delta Architectural Framework," is a set of modeling tools for Digital Engineering aiming to enable collaboration between roles such as Enterprise Architects, Solution Architects, and Business Analysts in designing the enterprise by creating high-quality models that can be transformed into artifacts such as code and documentation.

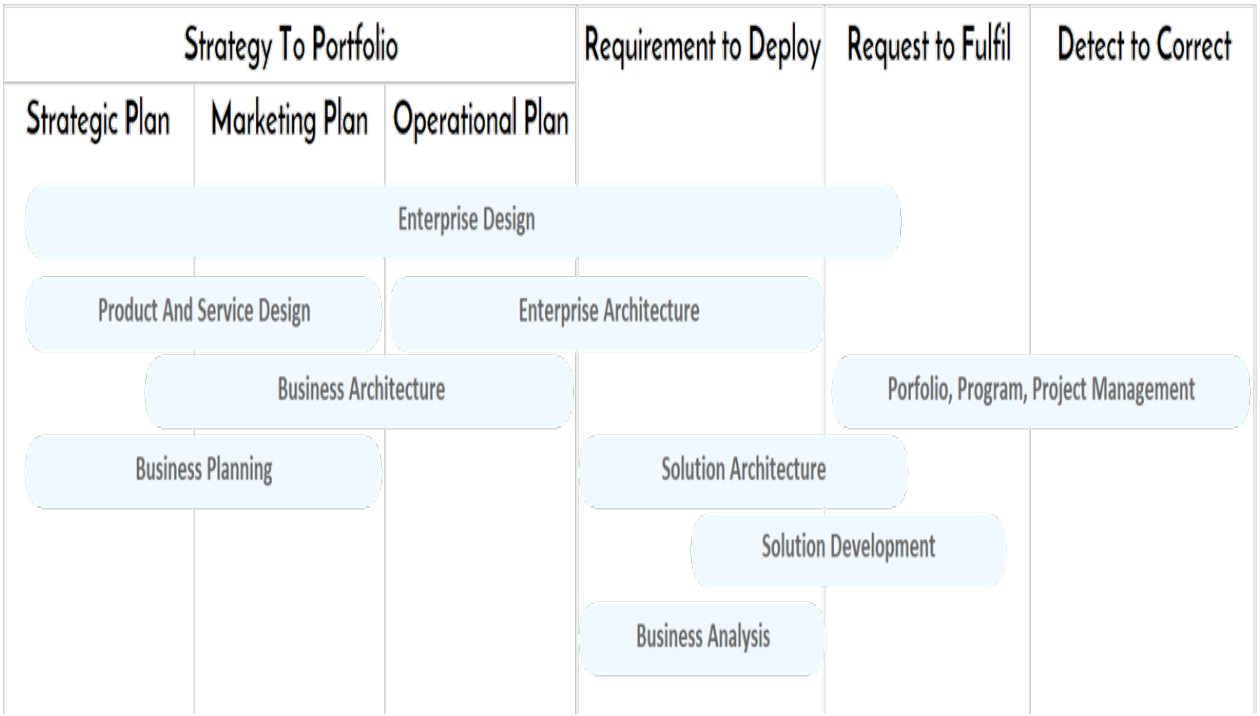


Figure 1: Value Streams covered by DAF

The diagram shows the classic gaps between strategy, design, and execution. DAF may accelerate the delivery of projects by supporting collaboration of different roles and by creating traceability between strategy and execution. In fact, it provides the metamodel to close these gaps, ensuring that strategic plans are directly reflected in architectures, architectures are transformed into solutions, and solutions generate tangible deliverables with traceability back to strategy.

The DAF meta-model is based on industry standards such as UML, ArchiMate, TOGAF, RUP, SPEM and best practices for requirement management, and offers a baseline of 75 concepts, 99 relationships and 38 enumerations that can be customized to meet specific needs. While DAF can be used on its own, it is not intended to represent a new standard but rather serves as an accelerator that allows for quick tailoring of existing standards.

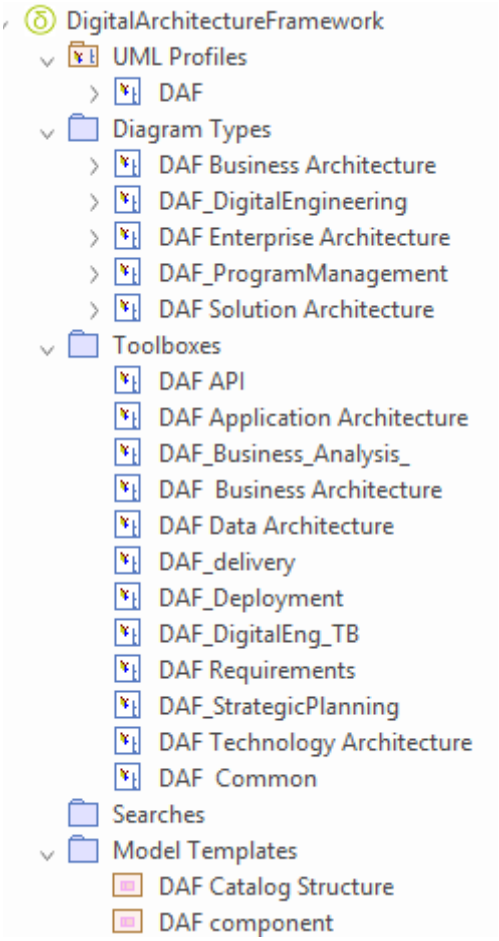


Figure 2: Content of the DAF MDG

DAF is currently implemented as MDG technology in Sparx Enterprise Architect, including:

- UML profile
- A set of customized icons
- Shape scripts
- multiple Toolboxes, organized by roles
- Custom diagrams, organized by roles
- Model patterns, accelerating the creation of DAF diagrams.
- Model Template with the structure of DAF Catalogs
- Quick linker
- A set of 100+ diagram examples showing how to use DAF to cover Enterprise Architecture, Solution Architecture, Design Architecture and Business Analysis. Those examples include all the diagrams in the TOGAF 9.1 specification.
- A structure based on TOGAF to host an enterprise repository
- A set of scripts, performing Model-to-Model (M2M) transformations
- A set of rules for model validation implemented in the EA Validator

- A semantic model of the metamodel (RDF/OWL with SHACL shapes), generated from the Sparx model

2 License

- DAF is Open Source, released under the Creative Commons
- 

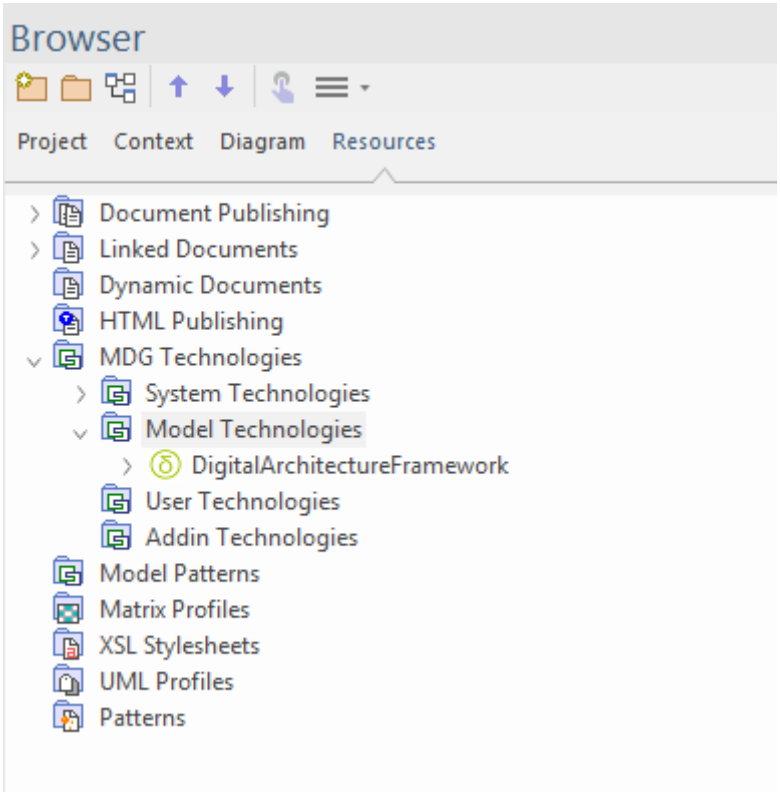
- Digital Architecture Framework © 2026 by Giu Platania is licensed under Creative Commons Attribution 4.0 International
- This license requires that reusers give credit to the creator. It allows reusers to distribute, remix, adapt, and build upon the material in any medium or format, even for commercial purposes.

3 Project

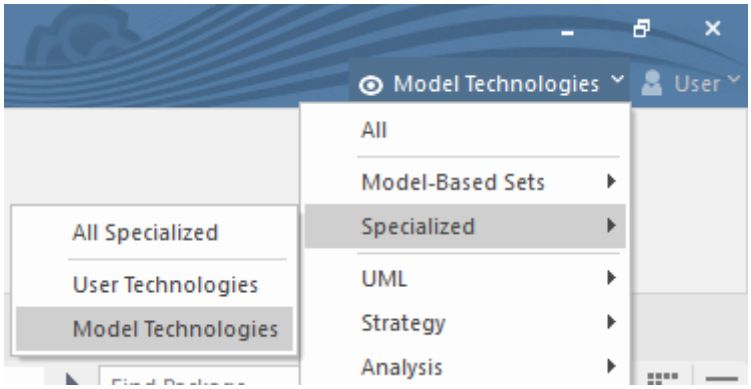
DAF is maintained in GitHub: Digital Architecture Framework GitHub repository

4 Installation

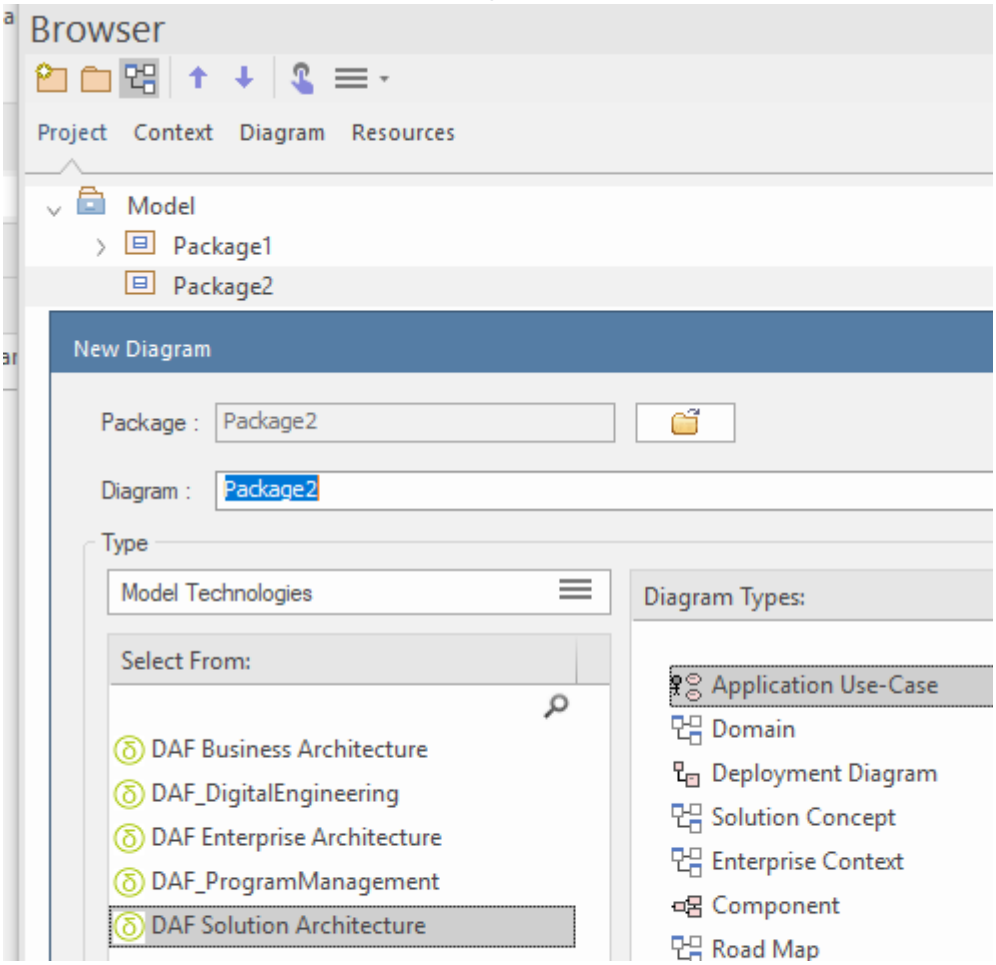
1. **Obtain the DAF MDG file**: Ensure you have the XML file defining the DAF MDG Technology from the repository Digital Architecture Framework releases. This file contains the definitions for stereotypes, toolboxes, profiles, and diagram types.
2. Save the file to an accessible directory on your computer.
3. Launch **Sparx Enterprise Architect**.
4. Open or create a project where you want to use the DAF MDG Technology.



5. In the browser, open the tab Resources and navigate to **MDG technologies> Model Technologies**, right click and select **Import Technology**.
6. Locate and select the daf.xml file
7. Select **import to Model**
8. Click **OK** to load the MDG Technology into Sparx EA.
9. **Optional: Enable the DAF MDG Technology**
    - Go to **Specialize > Technologies > Manage Technologies**.
    - In the list of available MDG Technologies, find "DAF" and check the box to enable it.
    - Click **Close** to save your settings.
10. **Suggested:** set your perspective focus on DAF



11. Test a DAF diagram
    - Create a new diagram
        - Select one DAF Diagram from the options



5 What's new in DAF 7.0

DAF 7.0 is a major release. It changes datatypes, renames relationships and adds concepts, so models created with DAF 6.x must be migrated (see Migrating 6.x models below). The changes come from GitHub issue #4, found while validating a complete test model against the DAF 6.5.1 semantic model and checker rules.

In numbers: 75 concepts, 99 relationship types and 38 enumerations. Every relationship end has a multiplicity, and the QuickLink holds 396 rules.

5.1 No invented values

An absent value now stays unknown. DAF 6.x forced modelers to fill in many properties and supplied placeholder defaults; in analysis and simulation, those defaults read as confident but wrong data.

- Money, volume and duration properties use the new Quantity value type and are optional: dCapability.Cost, dApplicationComponent.Cost, dRole.Cost and #FTEs, dActor.Count, dOrganizationUnit.HeadCount, dInitiative.EstimatedTotalCost, dGoal.Value_amount and Value_Goal, dProduct.Price, dBusinessProcess.ProcessVolumetrics, dMeasurementIndicator.CurrentLevel and SatisfactionLevel.
- The defaults 1, 10, 10.00, 100 and 5 on those properties are removed.
- RetireDate is a Date, 0..1, on dApplicationComponent, dBusinessProcess, dBusinessService, dPhysicalService and dProduct.
- All JSON Schema facets of dJSON_Type and dJSON_SchemaSubSet are optional.
- dInitiative.InitiativeDuration is a Duration; dInitiative.Benefit is removed.
- dRisk.likelihood is an Int32 from 1 to 100, optional and without default.
- Invalid defaults are corrected: dBusinessUseCase.isCore is false, and the stakeholder scales use Medium.

5.2 New concepts and value types

Concept	Purpose
Quantity (dQuantityValue)	A value with a unit: amount (Decimal), unit (ISO 4217 currency or UCUM code), per (rate period, e.g. P1Y), basis (e.g. per FTE), asOf and kind (Actual, Estimate, Assumption).
Cost Period (dCostPeriod)	Spend of an initiative in one period: periodStart, periodLength, amount and costNature (Capex, Opex). dInitiative.costProfile is an ordered list of cost periods.
Contract (dContract)	Commercial terms: pricingBasis, billableUnit, tariff (Quantity), effectiveFrom and effectiveTo, renewal (Duration) and the service-level properties. Linked to products (ContractRelatedProduct) and actors (dContractServesActor).

M3 gains the primitive types Decimal (exported as xsd:decimal) and Duration (ISO 8601, e.g. P1M, exported as xsd:duration). New enumerations: dCostNature, dEventType, dEventNature and dQuantityKind.

5.3 Events, KPIs and goals

- dEvent is both a trigger and an effect. It has eventType, nature (Recurring, Possible, Occurred), frequency, likelihood, startDate, endDate, duration and owner (a dRole); all are optional. ID, Category and the free-text Owner are removed.
- Four relationships form the flow between events and processes: dEventTriggersProcess (now to several processes), dProcessResultsInEvent, dProcessFlowsToProcess and dEventLeadsToEvent.
- KPIs can drive each other: dMeasurementIndicatorImprovesMeasurementIndicator and dMeasurementIndicatorWorsenMeasurementIndicator.
- A goal can target KPIs directly: dGoalTargetsdMeasurementIndicator.
- dActor.Count holds a population, e.g. the number of customers in a segment.

5.4 Relationships and multiplicities

Every relationship end now has a multiplicity of the form n, n..m or n..*. Ends that blocked realistic models are opened, for example: a goal can have several objectives, a KPI can measure several capabilities, a role can belong to several organization units, and a business service can use several data entities.

Misspelled names are corrected:

DAF 6.x	DAF 7.0
dApplicationComponentRealizesogicalComponent	dApplicationComponentRealizesLogicalComponent
dApplicationAgregatesFunction	dApplicationAggregatesFunction
dControllerControlIsModel	dControllerControlsModel
dApplicationIRunsInNode	dApplicationRunsInNode
dRiskIRelatedToRisk	dRiskIsRelatedToRisk
dBusinessAreadHasCapability	dBusinessAreaHasCapability
dRisk.Likehood	dRisk.likelihood
dVolume.MounthPath	dVolume.MountPath
dSystem.plattform	dSystem.platform

DAF 6.x	DAF 7.0
Madium, Tollerate, unknow, UPD	Medium, Tolerate, unknown, UDP

5.5 Enumerations

- Enumerations that were defined twice are merged, so each member is exported once.
- MimeTypes holds 12 generic IANA media types instead of GitHub-specific ones; dAPI.consumes and produces have no default.
- ArchitectureType adds s390x and ppc64le.

5.6 Model checks

- Check 009 (a function should be associated with an Application) follows the metamodel direction, dApplicationComponent aggregates dApplicationFunction.
- Check 002 reports elements still in status Proposed, and is renamed Elements with the EA status Proposed.
- Check 013 accepts components whose status is not set.
- Rules that name a renamed relationship accept both the 6.x and the 7.0 name until a model is migrated.

5.7 MDG technology

- UML profile and MDG technology (technology/DAF.xml) regenerated from the metamodel.
- Toolboxes: every stereotype is on at least one toolbox page; Contract, Cost Period and Quantity are added.
- Icons and shape scripts for dContract, dCostPeriod and dQuantityValue.
- The QuickLink carries the 7.0 rules.
- Semantic model: technology/rdf/DAF-semantic-model.ttl (RDF/OWL with SHACL shapes), generated by M3_DAF_Semantic_Model 1.5.0.

5.8 Migrating 6.x models

Import the 7.0 MDG technology into the model, then migrate the elements and connectors. The migration converts values only where the conversion is certain; anything else stays empty and is reported for a person to review. For example, a number without a unit becomes a Quantity only when its unit is known, and a value equal to a removed 6.x default is flagged. Renamed relationships and properties are mapped to their 7.0 names. The migration is tracked in GitHub issue #4 (item 28).

6 Concepts

6.1 From Reality to Abstraction

DAF is a metamodel (M2) that describes the elements of a Model (M1) that is, in turn, an abstraction of a real system (M0).

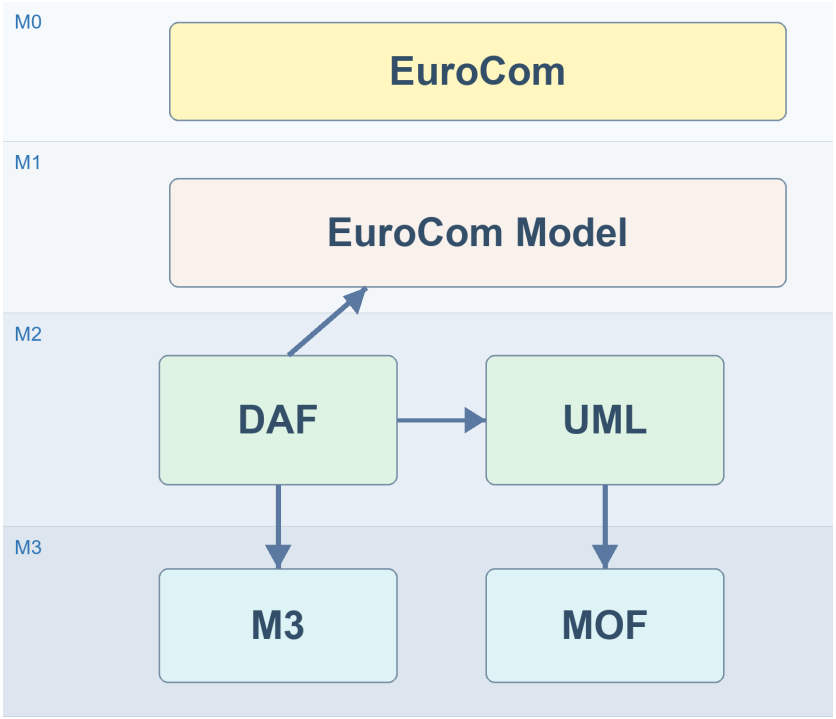


Figure 2: The hierarchy in this diagram follows the MDA principle of abstraction, where each level (M0 to M3) adds structure and abstraction to the level below, ensuring a consistent and scalable modeling approach.

- **M0: EuroCom:** The M0 layer represents the real-world organization or domain of interest, here as "EuroCom." This is the concrete instance of the domain that the models aim to represent, analyze, and improve. It consists of actual data and entities in the operational environment.
- **M1: EuroCom Model**: The M1 layer contains the model of the M0 domain. In this case, the "EuroCom Model" represents an abstraction of the EuroCom organization, describing its structure, behavior, or other aspects at a higher abstraction level. The M1 model acts as a blueprint that specifies how the M0 system is represented.
- **M2: DAF and UML**: The M2 layer defines the metamodels that describe the structure and semantics of the models in the M1 layer The arrow from DAF to UML indicates influence, highlighting how DAF integrates concepts from UML.
- **M3: M3 and MOF**: At this level, the fundamental rules and concepts for defining modeling languages are established. The M3 layer represents the meta-metamodels that describe the foundation for defining metamodels like UML or DAF. M3 is a custom meta-metamodel that can describe DAF or other metamodels. M3 is used instead of MOF (Meta-Object Facility): A standard meta-metamodel defined by the OMG to provide a formal foundation for modeling languages like UML.

6.2 From the Model to a View

There is a formal way of separating the idea of “one model” from the way we look at it, develop it, and communicate it.

Metamodel	Model	Layers	Viewpoint	View
A model documenting the elements used in a specific area of interest and their relationships.	A set of entities that describes the area of interest and its relationships.	A level of abstraction within the considered model. There are three layers in the DAF model.	A way of looking at the underlying model for a particular type of observer. There are 12 viewpoints in the DAF model.	What is seen from a viewpoint. Diagrams, matrices, and catalogs are types of views.

The Digital Engineering approach aims to enable the separation of concerns across different models, metamodels and viewpoints, allowing for a more flexible, maintainable and understandable representation of the system, making it possible for different stakeholders to focus on the aspects of the system that are relevant to them.

A "**metamodel**" is a model … of a model, which defines the structure and constraints of the models that conform to it. It describes the concepts, properties, and relationships that a model can contain, as well as the constraints and rules that apply to them. Metamodels are often expressed in formal languages, such as MOF or eMOF.

The concept of a "**model**" refers to a representation of a system or part of it in a specific format that adheres to the metamodel (e.g. UML or ArchiMate). These models can be used to define the architecture of a system. The ISO/IEC/IEEE 42010 standard defines "*architecture*" as "*the fundamental organization of a system, embodied in its components, their relationships to each other and the environment, and the principles guiding its design and evolution*."

.

A "**Model Layer**" is a set of models that conform to a specific metamodel and are used to represent a specific aspect of the system being modeled. For example, one model layer could be used to represent the logical architecture of the system, while another could be used to represent the physical deployment of the system.

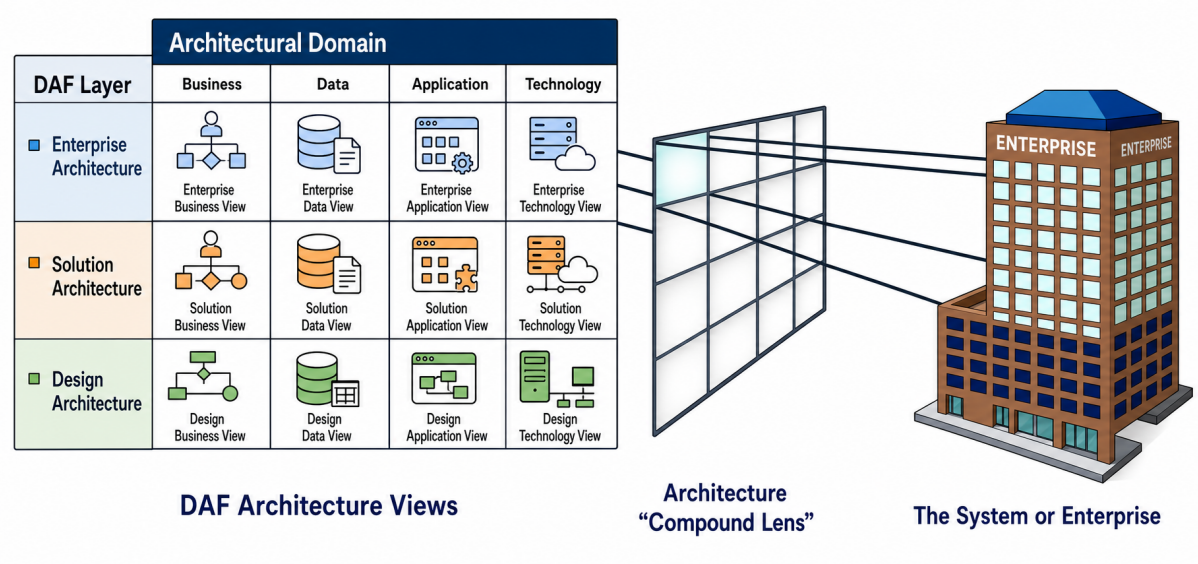


Figure 3: Relation between IT view and Enterprise

A "**Viewpoint**" is a set of views that have a common purpose or perspective. It can be related to specific domain, such as security or performance, and defines a specific set of views and model elements to be used to capture the relevant information from the system.

A "**View**" is a perspective or a specific part of a model that is relevant to a particular audience, such as stakeholders or developers. Each view provides a specific representation of the system, highlighting different aspects of the system and omitting the less important details.

6.3 DAF Metamodel

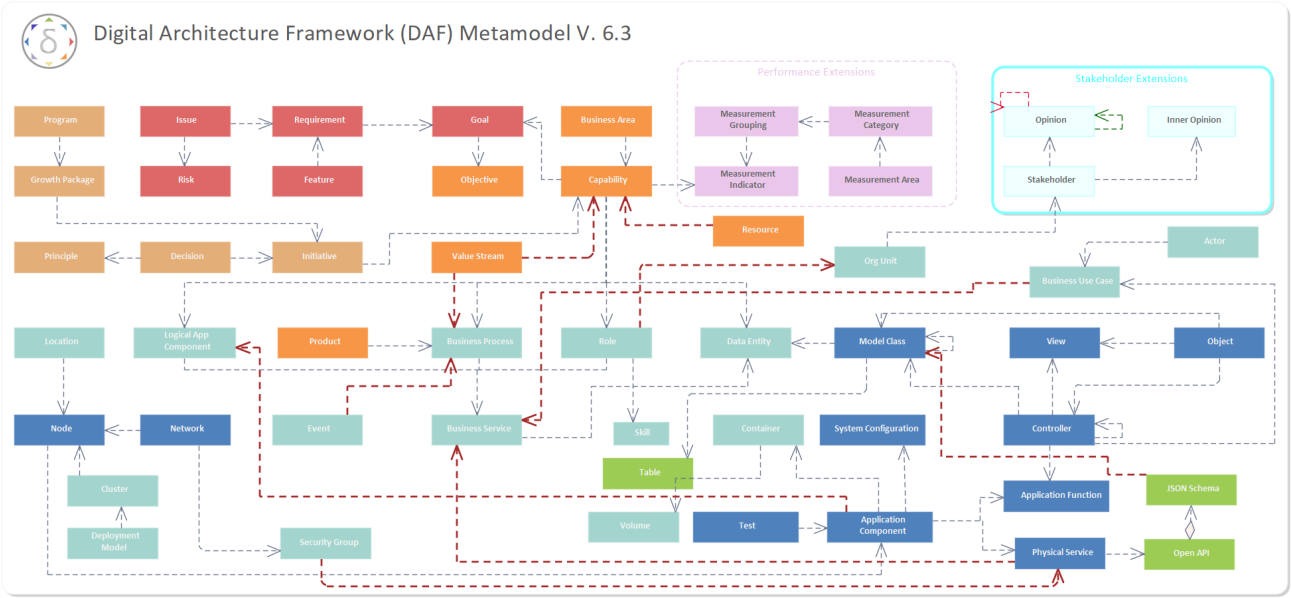


Figure 4: DAF Metamodel

The DAF Metamodel supports Enterprise Architecture, Solution Architecture, Design Architecture and more. See the description of the DAF layers below.

6.4 Example Model Instance

Each element of the Metamodel can be instantiated into a concrete model's element.

The following diagram shows an example of a DAF Metamodel instance

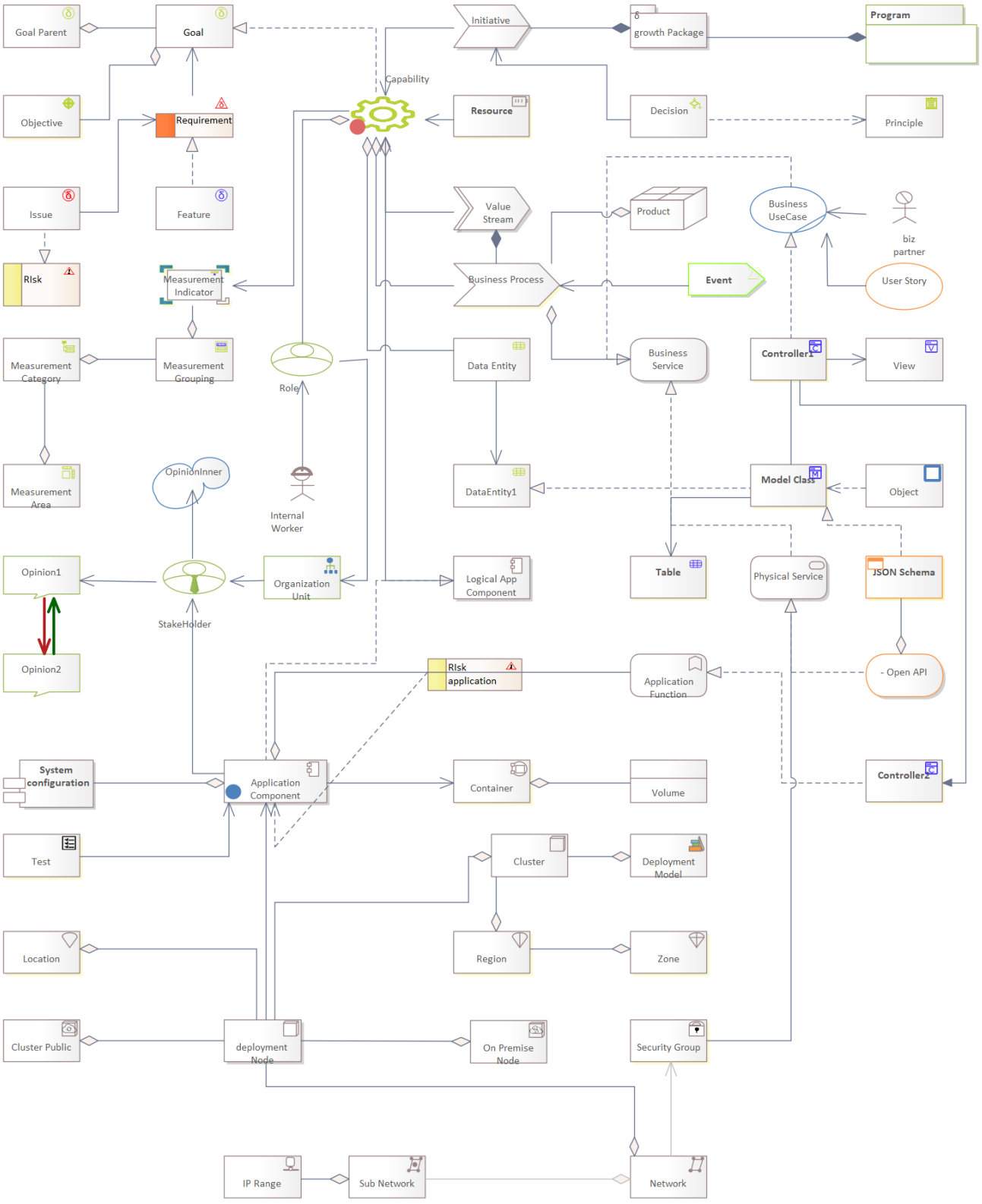


Figure 6: DAF Example Instance

DAF supports the Sparx EA quick linker that helps modelers to create only valid connections. Models can be also validated using the Checker (see below)



6.5 Layers

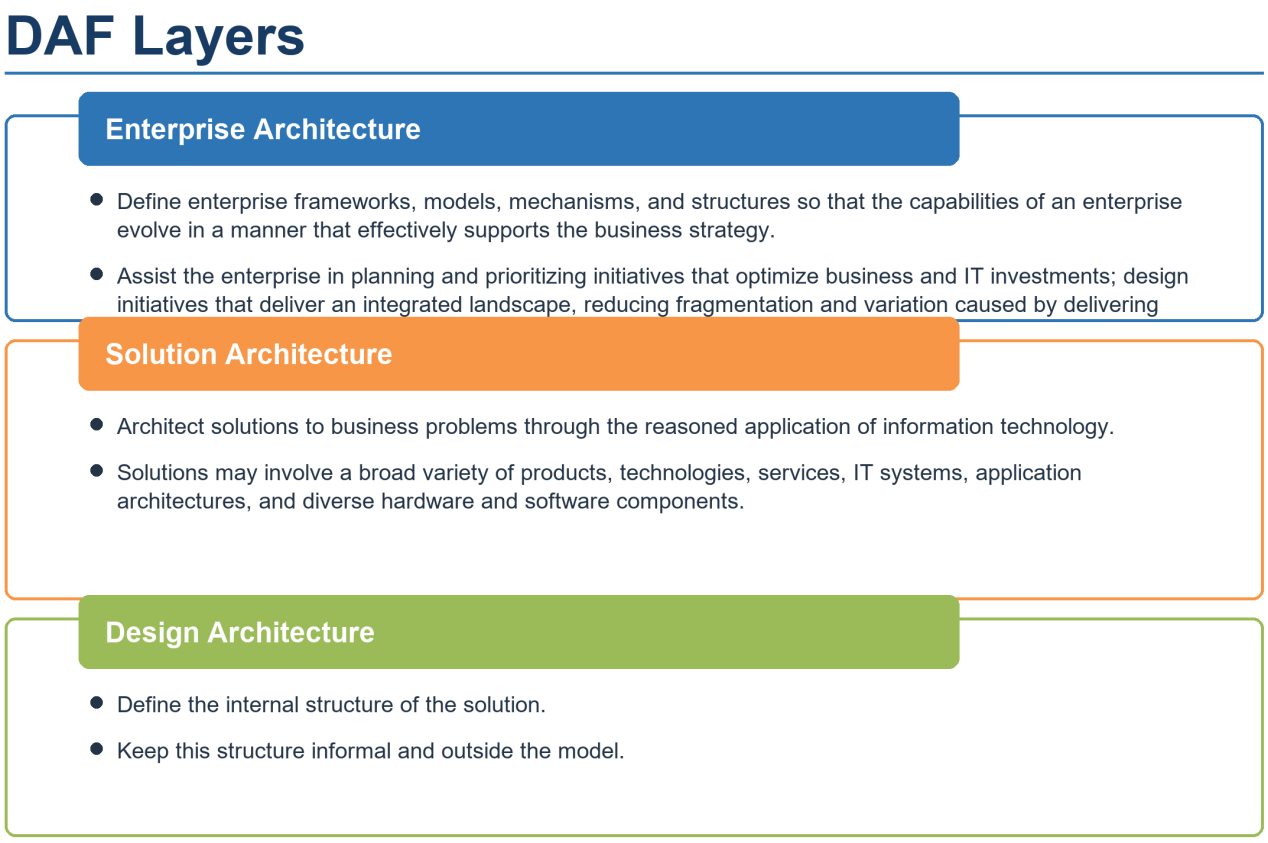


Figure 7: DAF Layers

DAF is divided into three levels of abstraction: Enterprise, Solution, and Design Architecture. Each of these levels is further broken into specific domains (see below).

6.5.1 Enterprise Architecture

- This Layer covers logical considerations. Viewing the system in terms of roles participating in processes by using resources: Who, How, and What. It corresponds to the Computational Independent Model (CIM).
- It defines enterprise frameworks, models, mechanisms, and structures so that capabilities of an enterprise evolve in a manner that effectively supports the business strategy
- Assisting the enterprise to plan and prioritize initiatives that optimize business & IT investments; designing initiatives that deliver an integrated landscape to reduces the fragmentation and variation that is the natural result of delivering solutions in an autonomous manner.
- The Enterprise Architecture level focuses on the logical considerations of the system, such as the roles, processes, and resources involved. This abstraction level helps the enterprise plan and prioritize initiatives that optimize business and IT investments, and design initiatives that deliver an integrated landscape to reduce fragmentation and variation.

6.5.2 Solution Architecture

- The Solution Architecture layer connects to the previous and translates the logical model into an operational one by describing the elements of the system that supports the business and assigns processes to them.
- It aims as definition of solutions to business problems through the reasoned application of Information Technology.

- They may involve the application and integration of a broad variety of products, technologies and services, various IT systems and applications architectures, and diverse hardware and software components.

6.5.3 Design Architecture

- This Layer designs the components of the solution, taking in account the different supporting technologies.
- The definition of the internal structure of the solution.
- Generation of Code

6.6 Domains

The abstraction layers are further organized into domains. A domain represents a **vertical organization** of concepts, incorporating related concepts at different levels of abstraction. These domains serve to organize and structure the various components and elements of the architecture in a logical and meaningful way.

The foundational domains in DAF are:

- Business
- Data
- Application
- Technology

Business Architecture	Data Architecture	Application Architecture	Technology Architecture
A representation of holistic, multi-dimensional business views of: capabilities, end-to-end value delivery, information, and organizational structure; and the relationships among these business views and strategies, products, policies, initiatives, and stakeholders. *TOGAF*	*A description of the structure and interaction of the enterprise's major types and sources of data, logical data assets, physical data assets, and data management resources.* *TOGAF*	*A description of the structure and interaction of the applications as groups of capabilities that provide key business functions and manage the data assets.* *TOGAF*	1. *A technology building block. A generic infrastructure technology that supports and enables application or data components (directly or indirectly) by providing technology services.* 2. *An encapsulation of technology infrastructure that represents a class of technology product or specific technology product.*

Figure 5: Foundational DAF domains, inspired by TOGAF

6.6.1 Business

This domain covers the business-specific concepts, such as business processes, services, and functions that an organization offers to its customers. It focuses on understanding the business objectives and strategies, and how the IT solutions can support those goals.

6.6.2 Data

This domain covers information-related concepts, such as data models, data architecture, data governance, data security, and data management. It's focused on understanding the organization's data requirements, and how the IT solutions can support data management and access.

6.6.3 Application:

This domain covers software-related concepts, such as application architecture, application integration, and service-oriented architecture. This domain is focused on understanding the organization's application requirements, and how the IT solutions can support application deployment and management.

6.6.4 Technology

This domain covers technology-related concepts, such as infrastructure, network, security, and cloud. This domain is focused on understanding the organization's technology requirements, and how the IT solutions can support infrastructure and technology management.

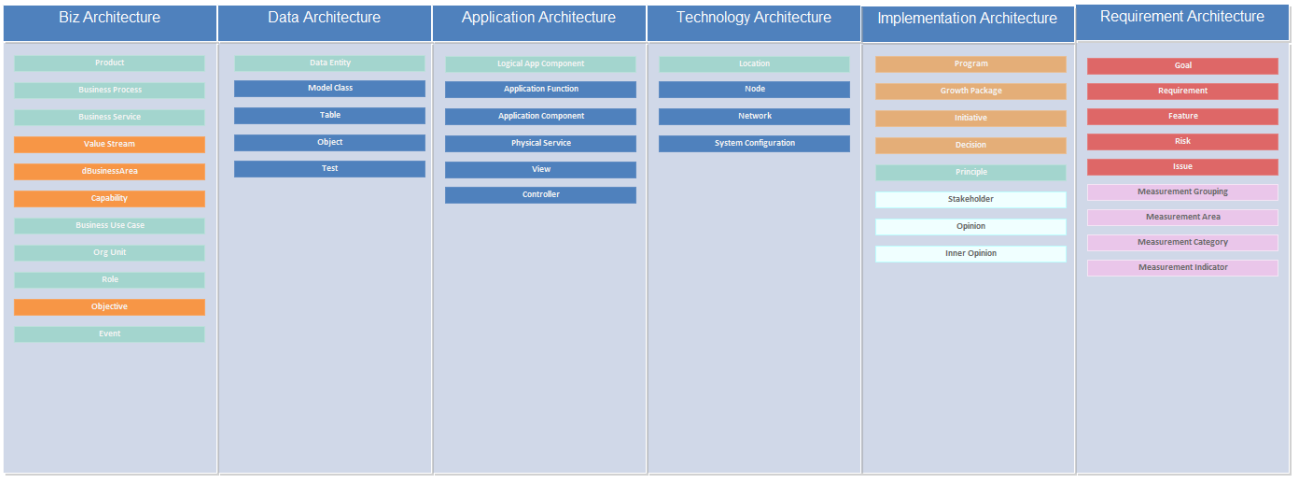


Figure 6: Domains and concepts of DAF

6.6.5 Extension Domains

Additional Domains can be also used with DAF when required.

Those include Strategy, Implementation and Requirements.

6.7 Viewpoints and Views

The intersection of a Layer and Domain creates a Viewpoint, addressing specific concerns about a system of interest. A Viewpoint is a way of looking at a system from a specific perspective and addressing specific concerns related to that system. In the context of DAF, a **Viewpoint** is the intersection of a Layer and a Domain.

Each Layer, such as Enterprise, Solution, and Design Architecture, provides a different level of abstraction and focus, while each Domain, such as Business, Data, Application, and Technology, represents a specific area of concern. By intersecting these two concepts, a Viewpoint is created that addresses specific concerns of the system-of-interest from the perspective of that Layer and Domain. For example, a Business Viewpoint within the Enterprise Layer would focus on the logical considerations of how the system supports the business strategy, while a Data Viewpoint within the Design Layer would focus on the design of the data components of the solution.

The use of Viewpoints allows for a more targeted and granular approach to analyzing and designing a system, as different stakeholders can look at the same system from different perspectives.

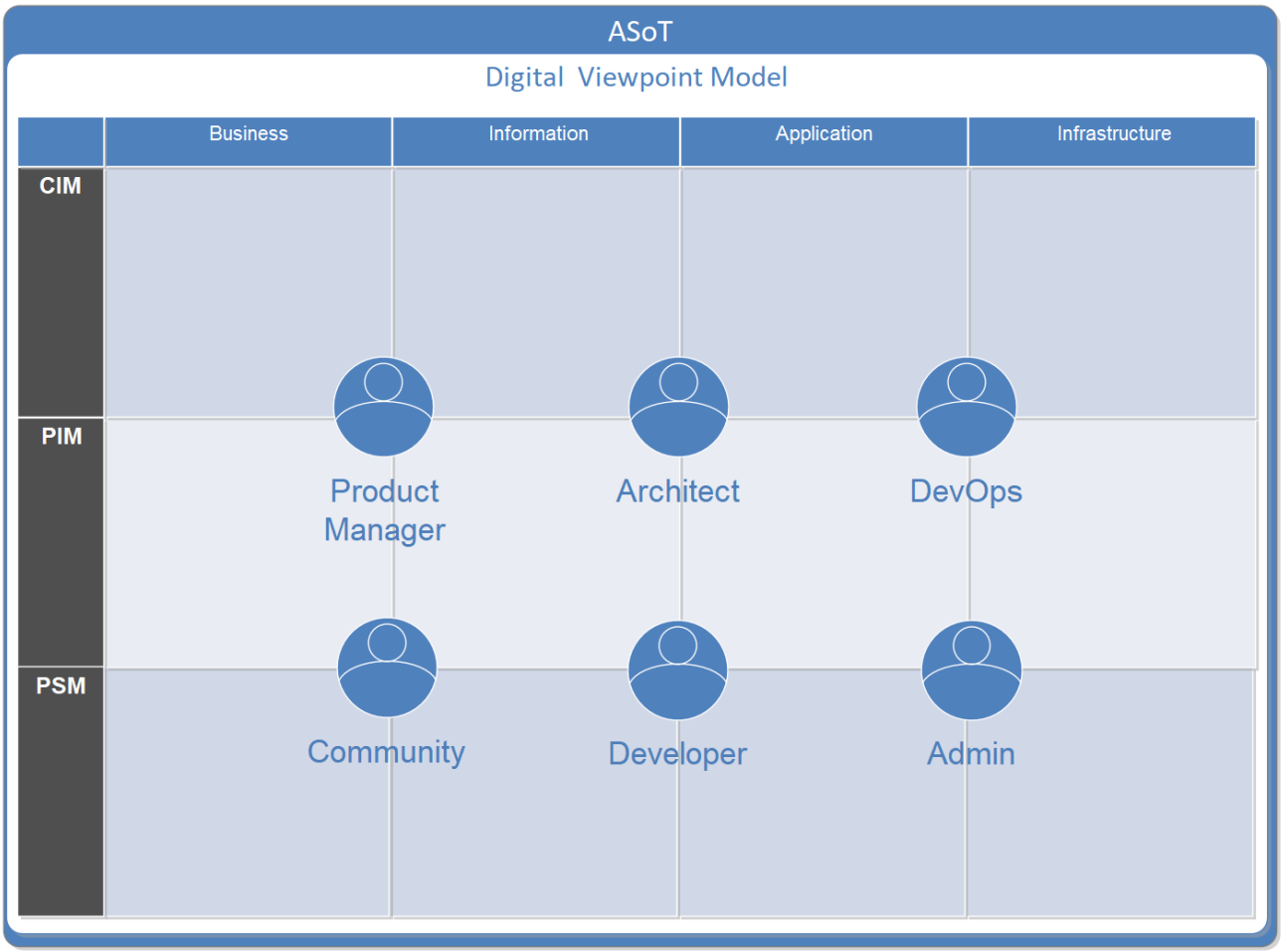


Figure 7: Digital Viewpoint Model

DAF features 12 basic viewpoints

Figure 8: example of viewpoint and their stakeholders

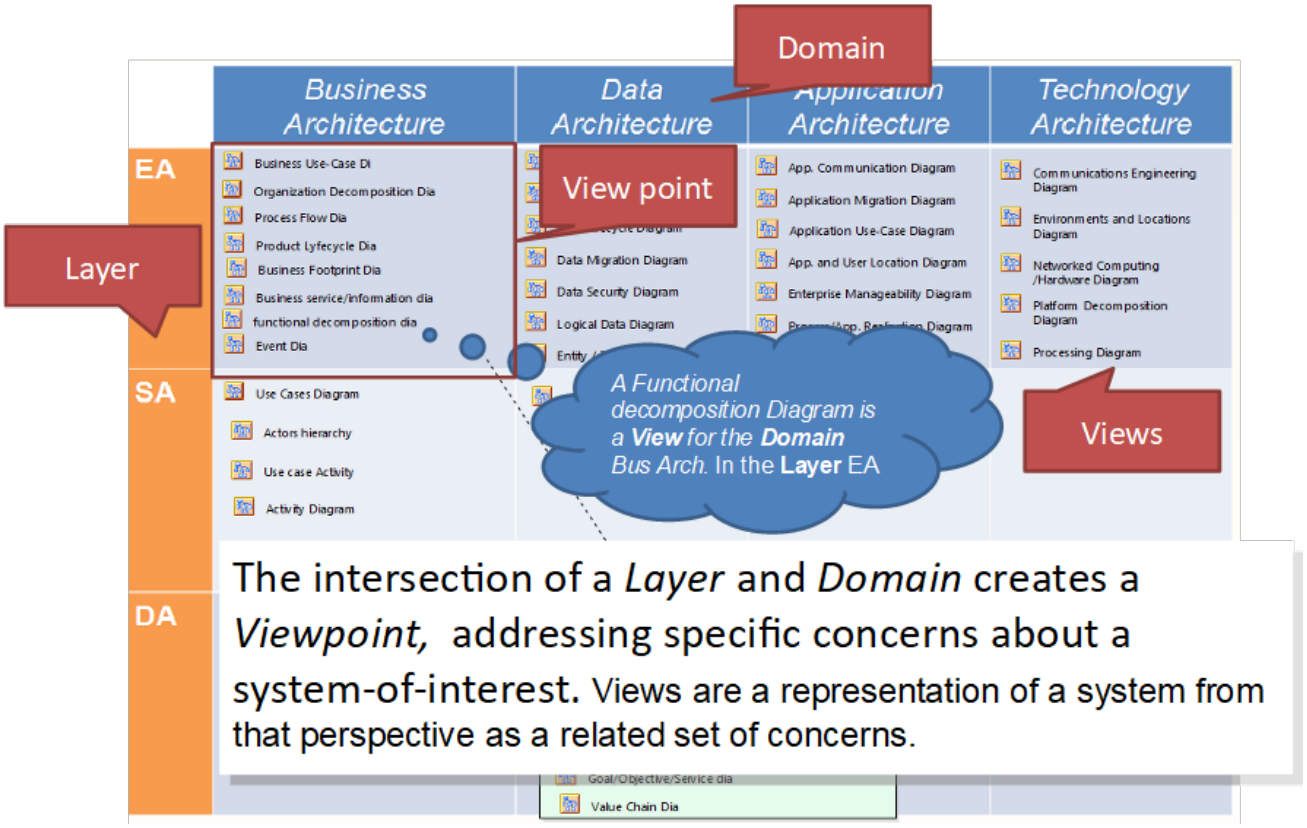


Figure 8: DAF Viewpoints

Views are representations of a system from that perspective as a related set of concerns.

6.7.1 DAF Concepts Organized in Viewpoints

Each of the concepts of the Metamodel belong to a specific Viewpoint. As such, it's easier to assign responsibility to certain roles.



Figure 9: DAF Concepts Organized in the Viewpoints

6.7.2 Views and work products

DAF supports the SPEM concept of Deliverable.

A **Deliverable** is created when a **role** performs an activity within a process that produces one or more views. Each view is built from the DAF entities stored in the repository catalog. Multiple views together form a complete deliverable.

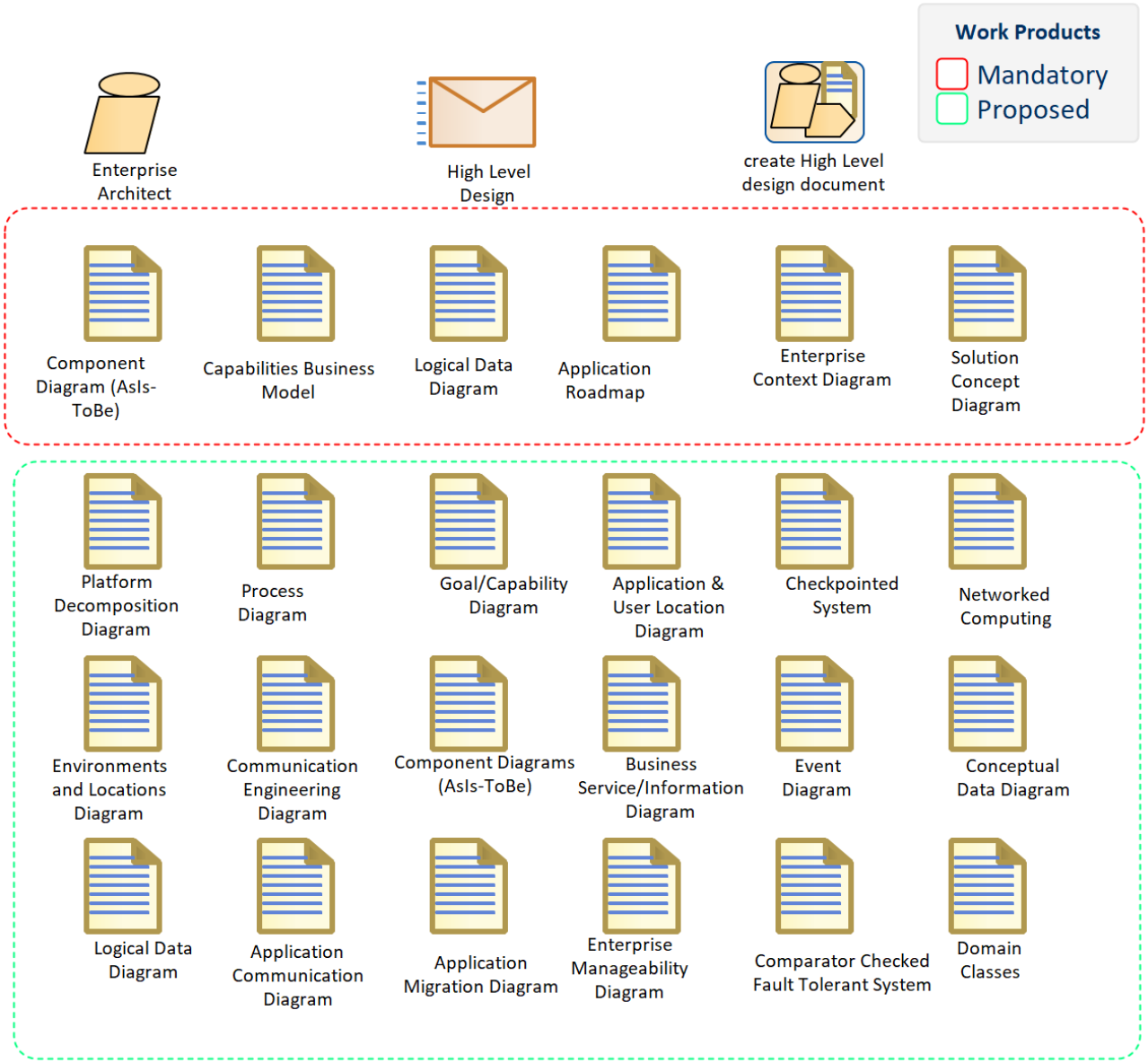


Figure 10: SPEM View with the Work Products That Compose the High-Level Design Deliverable

In the example above, the Enterprise Architect performs the activity *Create High-Level Design Document*. The result is the **High-Level Design** deliverable. This includes mandatory work products, such as the *Component Diagram* (As-Is/To-Be) and the *Capability Business Model*. Depending on the scope, it may also include optional work products, like the *Platform Decomposition Diagram* or the *Process Diagram*.

By its nature, a deliverable is like a 2D photo: it presents a static view that does not necessarily expose all implications or dependencies. However, the underlying model contains the full traceability, which can be retrieved to perform impact analysis and uncover relationships not immediately visible in the deliverable itself.

6.8 Relationships

Name	Meaning	Notation
Association	*"has"*	
Aggregation	*"shared with"*	
Composition	*"owned-by"*	
Dependency	*"uses"*	
Generalization	*"is-kind-of"*	
Realization	*"implements"*	

Figure 11: Basic UML relationships

DAF relationships are extending fundamental UML relationships:

- Association: A relationship that models a bi-directional semantic connection among instances
- Aggregation: A special kind of association that models a "whole-part" as shared relationship
- Composition: non-shareable, non-transferable ownership
- Dependency: A relationship that signifies that a single or a set of model elements requires other model elements
- Generalization: A taxonomic relationship between a more general element and a more specific element
- Realization: A specialized abstraction relationship between two sets of model elements, one representing a specification (the supplier) and the other represents an implementation of the latter (the client)

In the DAF metamodel, inheritance uses Generalizations, transitions of abstraction layers are represented with stereotyped Realization, strong connections between elements of the same abstraction level with stereotyped **Associations**, weak relationships with Dependencies; strong hierarchical relationships with **Compositions**, weak ones with **Aggregations.**

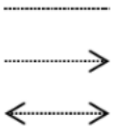
Name	Meaning	Notation
Triggering	Describes a temporal or causal relationship between elements.	
Flow	Transfer from one element to another.	
Assignment	Expresses the allocation of responsibility, performance of behavior, or execution.	
Serving	Models that an element provides its functionality to another element.	
Access	Models the ability of behavior and active structure elements to observe or act upon passive structure elements.	
Influence	Models that an element affects the implementation or achievement of some motion element.	

Figure 12: Additional ArchiMate relationships

6.8.1 Relationship Strategy

The DAF Metamodel is designed to support the concept of **encapsulation**.

Encapsulation is a key concept in modeling that refers to the practice of enclosing concept and behaviors within a single entity, so as to protect its internal state from external interference. This helps to maintain the integrity of the model by controlling the flow of information and avoiding circular references. In object-oriented modeling, encapsulation allows objects to interact with one another without being dependent on their implementation, creating a level of abstraction that makes the system more flexible and maintainable.

Circular references in modeling refer to the situation where two or more models or elements within a model refer to each other in a circular manner. This creates a situation where one element depends on another, which in turn depends on the first, leading to a never-ending chain of inter-dependencies. Circular references are often considered problematic in modeling because they can lead to ambiguities and inconsistencies in the models, making it difficult to analyze and understand the relationships between elements. They can also cause problems with automated analysis and interpretation tools that rely on the models.

Instead of circular references, **derived relationships** are used.

A derived relationship refers to a relationship that is obtained or computed from other relationships or information in the model. It is not directly specified in the model but is instead derived from existing elements and their properties and relationships. Derived relationships are often used to simplify a model and make it more manageable by abstracting complex relationships and showing only the essential relationships between elements. These relationships may also be used to generate additional information about the system being modeled or to support certain modeling activities such as impact analysis, traceability, and consistency checks.

7 Detailed Documentation

The following is the complete documentation of the DAF metamodel

7.1 DAF Concepts Overview

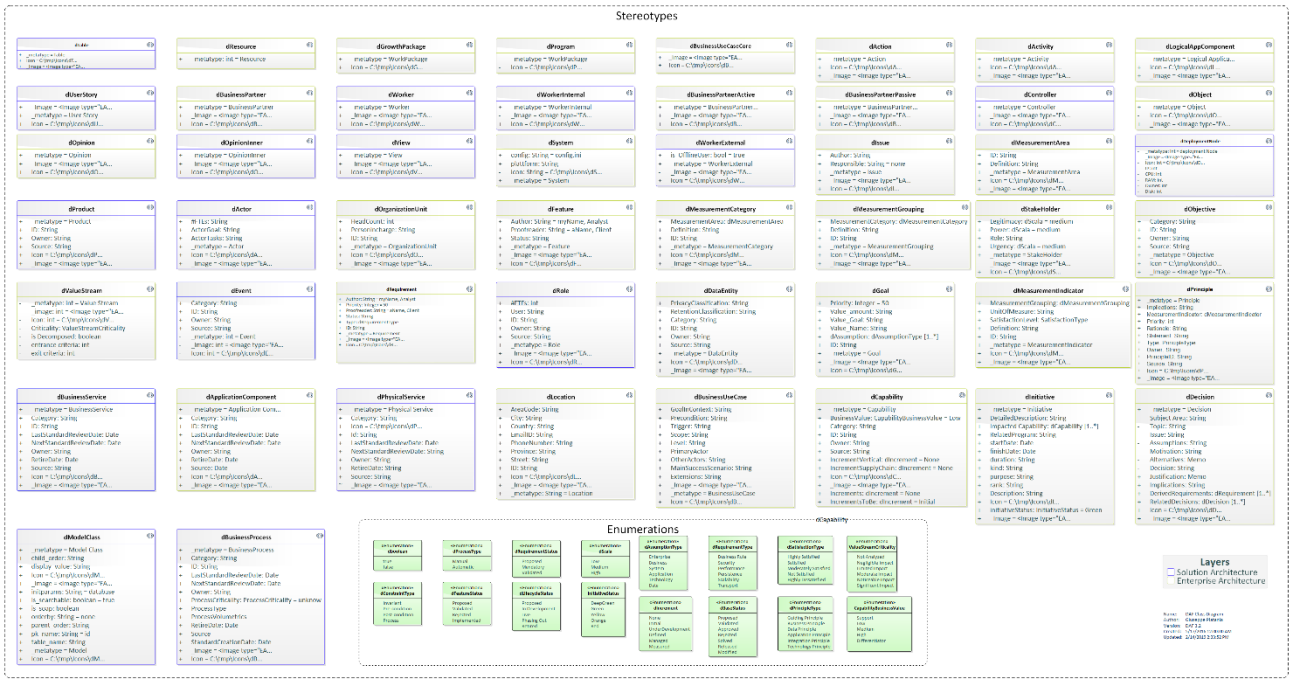


Figure 13: DAF Concepts and Properties

DAF version 7.0 supports 75 concepts (implemented as UML Stereotypes), each of those has several properties (expressed as UML Tagged Values). The table is generated from the DAF semantic model (technology/rdf/DAF-semantic-model.ttl).

Domain	Technical Name	Human Name	Documentation
			an organization of Business interests, can be decomposed.
Strategy	dBusinessArea	Business Area	
Strategy	dCapability	Capability	Represents an organizational capability to achieve business outcomes.Can be current available in an Organization Unit (or) A business-focused outcome that is delivered by the completion of one or more work packages (by increasing the capability Maturity).
			A place where business activity takes place and can be hierarchically decomposed.
Strategy	dLocation	Location	
Strategy	dMeasurementArea	Measurement Area	Represents areas of organizational or business performance measurement. The organization or Business units performance objective is captured as measurement area.
			Categorizes measurements within a measurement area.
Strategy	dMeasurementCategory	Measurement Category	

Domain	Technical Name	Human Name	Documentation
			a groups of Measurement Indicator that can be aggregated into Categories
Strategy	dMeasurementGrouping	Measurement Grouping	
			The specific measure captured for a Organization or Business Unit which can be measured and quantifiable. A measurement is a Key Performance indicator
Strategy	dMeasurementIndicator	Measurement Indicator	
			The Objective is a non material achievement, other than a Goal, the objective has no specific value to be measured. it's a time-bounded milestone for an organization used to demonstrate progress towards a goal.
Strategy	dObjective	Objective	
			A view or judgment formed about a Topic. That is formulated in an explicit fashion. Captures explicit stakeholder judgments on a topic.
Strategy	dOpinion	Opinion	
			Captures implicit stakeholder opinions related to a topic. A Stakeholder’s opinion that is relevant for the topic but is not formulated explicitly.
Strategy	dOpinionInner	Inner Opinion	

Domain	Technical Name	Human Name	Documentation
Strategy	dOrganizationUnit	Org Unit	Defines organizational units responsible for roles, can be decomposed
Strategy	dPrinciple	Principle	Defines qualitative statements guiding architecture intent.
Strategy	dProduct	Product	Encapsulates outputs produced by business processes.
Strategy	dResource	Resource	a type of Asset supporting a Business capability
Strategy	dSkill	Skill	an ability that some role possesses with various degree

Domain	Technical Name	Human Name	Documentation
Strategy	dStakeholder	Stakeholder	Represents any party or force influencing the domain of interest. Any person or force that have an interest in the considered domain. This is typically indicated by name, can be connected with a Role in the organization. A stakeholder has open and Inner opinions that are relevant for the subject matter.
Strategy	dValueStream	Value Stream	A representation of an end-to end collection of value-adding Business Processes that create an overall result for a customer, stakeholder or end user. A sequence of activities an enterprise undertakes to deliver on a customer request. More broadly, the sequence of activities required to design, produce, and deliver a good or service to a customer, and it includes the dual flows of information and material.
Business	dAction	Action	A specialized activity.
Business	dActivity	Activity	Represents business or system activities. A function or work carried out by the business user.
Business	dActor	Actor	Represents an entity (such as a person, system, or organization) that interacts with the system to achieve a specific goal. An actor typically performs actions within business Use cases and can be internal (e.g., employees) or external (e.g., customers or partner organizations), active and passive.

Domain	Technical Name	Human Name	Documentation
Business	dBusinessProcess	Business Process	A sequence of activities that produce a specific service or product supporting a particular business capability
Business	dBusinessService	Business Service	A repeatable activity; a discrete behavior that a building block may be requested or otherwise triggered to perform. A service provided by Business that achieve a business outcome in response to a request.
Business	dBusinessUseCase	Business Use Case	Encapsulates business processes producing value for an enterprise.
Business	dEvent	Event	Something that happens, inside or outside the organization, that triggers behaviour or results from it.
Business	dRole	Role	The usual or expected function of an actor, or the part somebody or something plays in a particular action or event. An actor may have a number of roles.

Domain	Technical Name	Human Name	Documentation
Requirement	dFeature	Feature	A Feature is a fact that set his related Requirement to be true
Requirement	dGoal	Goal	a Goal is a measurable scope that the organization wants to achieve. Can be hierarchically decomposed.
Requirement	dIssue	Issue	Am issue is a concrete problem that is affecting a business. Also, a fact that sets a Requirement to be false. (see also Risk)
Requirement	dRequirement	Requirement	A Business guide line about the Enterprise or the project. Requirements can be decomposed. A requirement is formulated in a SMART fashion and uses Moscow verbs (Must Have, Should Have, Could Have, and Would Like to Have.). Must Have: These are the requirements that are considered essential for the success of the initiative and must be meet for the system to function. They are often referred to as "hard" or "mandatory". requirements and are typically expressed in Boolean format as "true" or "1". Should Have: These are the requirements that are considered important, but not essential. They are typically expressed in Boolean format as "maybe" or "0/1" and are often used to prioritize the development of the system. Could Have: These are the requirements that are considered desirable but not essential. They are typically expressed in Boolean format as "false" or "0". They are used to identify potential enhancements to the system that can be added at a later date. Would Like to Have: These are the requirements that are considered less important and are often used to identify potential future enhancements to the system. In addition, the requirement should be SMART, which stands for Specific, Measurable, Achievable, Relevant, and Time-bound. SMART requirements are clear, concise, and easy to understand, and they

Domain	Technical Name	Human Name	Documentation
			provide a clear way to measure the success of the project.
Requirement	dRisk	Risk	a risk Identifies and assesses potential issue impacting other elements. It is a potential future Issue. Can be hierarchically decomposed.
Implementation	dDecision	Decision	Represents decisions and their impact on architecture or processes.
Implementation	dGrowthPackage	Growth Package	A set of actions identified to achieve one or more objectives for the business. A work package can be a part of an Initiative, a complete project, or a part of a Program. A Growth pack is a collection of Initiatives architected for consistency and aimed to be executed in the same period.
Implementation	dInitiative	Initiative	An activity carried out by a Business Unit or organization to achieve a particular goal or improve the Maturity of a certain capability.
Implementation	dProgram	Program	A group of Growth Packages that achieve a complex outcome for the organization

Domain	Technical Name	Human Name	Documentation
Implementation	dUserStory	User Story	Captures user-centric requirements expressed as stories.
Application	dApplicationComponent	Application Component	An application component is a self-contained unit. As such, it is independently deployable, re-usable, and replaceable. An application component performs one or more application functions. It encapsulates its behavior and data, exposes services, and makes them available through interfaces. Cooperating application components are connected via application collaborations. An application component may be assigned to one or more application functions. An application component has one or more application interfaces, which expose its functionality. Application interfaces of other application components may serve an application component. The name of an application component should preferably be a noun.
Application	dApplicationFunction	Application Function	Encapsulates functions supported by applications.
Application	dController	Controller	Encapsulates application behavior and flow logic.
Application	dLogicalAppComponent	Logical App Component	Encapsulates logical software components independent of implementation or technology.
Application	dPhysicalService	Physical Service	a technical implementation of a service that runs on physical or virtual infrastructure, providing specific functionalities, usually in the form of software applications, middleware, databases, or platforms. It is concerned with the deployment, operation, and management of the technology stack, ensuring that the

Domain	Technical Name	Human Name	Documentation
			service is available, performant, and reliable. Examples include web servers, APIs, databases, and network services.
Application	dTest	Test	Represents a mechanism for verifying or validating the functionality, behavior, or compliance of am application component . Tests can include automated or manual procedures designed to ensure alignment with requirements, detect defects, and confirm expected outcomes.
Application	dValue	Value	An attribute of a Node value type used in DAF. These values are persistent
Application	dValueRef	Value Ref.	A Node value type used in DAF that reference another Node Value, similar to a foreign Key. These values are persistent
Application	dView	View	A View used in DAF. Controller, Views and Associations define the application flow
Data	dModelClass	Model Class	Represent a class of the domain model, that can be converted into a Persistent domain class, must inherit from Node. It is the application's dynamic data structure

Domain	Technical Name	Human Name	Documentation
Data	dObject	Object	an Object is an instance of a class in a particular moment in runtime that can have its own state and data values.
Data	dTable	Table	Represents physical tables in a database schema.
Technology	dAPI	Open API	OpenAPI Specification is an open-source format and initiative for designing and creating machine-readable interface files that are utilized in producing, describing, consuming, and visualizing RESTful APIs and web services.
Technology	dAPIOperation	API Operation	Defines operations exposed by an API for interaction with external systems. Also called endpoints.
Technology	dAPIParameter	API Parameter	Defines parameters in API operations.

Domain	Technical Name	Human Name	Documentation
Technology	dAPIResponse	API Response	Represents responses returned by API operations.
Technology	dCluster	Cluster	Represents a set of nodes working in the same Region, akin to ArchiMate’s Node concept within a server infrastructure.
Technology	dContainer	Container	Encapsulates application components and services, mirroring ArchiMate's concept of a Node, where an application runs.
Technology	dDataEntity	Data Entity	An encapsulation of data that is recognized by a business domain expert as a thing. Logical data entities can be tied to applications, repositories, and services and may be structured according to implementation considerations.
Technology	dDeploymentModel	Deployment Model	The root class for the description of a Deployments to roll out an application, service, or container to a set of resources. akin to ArchiMate's Technology Layer deployment views.

Domain	Technical Name	Human Name	Documentation
Technology	dDeploymentNode	Node	Represents a physical machine or virtual server that hosts services, applications, or containers, akin to ArchiMate's Node concept in the Technology Layer.
Technology	dIPRange	IP Range	Defines a range of IP addresses (CIDR) used within a network. This aligns with ArchiMate's concept of Infrastructure Service, providing connectivity.
Technology	dJSON_Attribute	JSON Attribute	Defines attributes in JSON data structures.
Technology	dJSON_Datatype	JSON Data Type	Represents a data type used within JSON structures, defining the format and constraints of data elements in a schema. JSON Data Types include standard types such as string, number, object, array, boolean, and null, ensuring consistency and validation in data exchange between systems.
Technology	dJSON_Element	JSON Element	Represents elements or nodes in JSON schemas.

Domain	Technical Name	Human Name	Documentation
Technology	dJSON_Schema	JSON Schema	The Schema Object allows the definition of input and output data types. These types can be objects, but also primitives and arrays. This object is based on the JSON Schema Specification Draft 4 and uses a predefined subset of it. On top of this subset, there are extensions provided by this specification to allow for more complete documentation
Technology	dJSON_SchemaSubSet	JSON Schema Sub Set	Represents subsets of JSON schema structures.
Technology	dJSON_Type	JSON Type	Represents data type definitions for structured information.
Technology	dNetwork	Network	The main communication medium between services and deployments, mapped to ArchiMate Communication Networks, facilitating inter-service communication.
Technology	dOnPremise	On Premise Node	an organization of Business interests

Domain	Technical Name	Human Name	Documentation
Technology	dPublicCluster	Public Cluster	A public cloud-based cluster, following the ArchiMate concept of Infrastructure Service provided by external cloud providers (Google, AWS, etc.).
Technology	dRegion	Region	A geographic area containing clusters and zones, aligned with ArchiMate's Location concept for representing deployment geography.
Technology	dSecurityGroup	Security Group	Controls and manages access and permissions between services, in line with TOGAF's Security Architecture for protecting digital assets.
Technology	dSubNetwork	Sub Network	A subset of a network, used to organize and isolate portions of the network. Equivalent to an ArchiMate Node, responsible for networking resources.
Technology	dSystem	System Configuration	A system configuration. Settings are modeled as attributes with default values, each attribute of the System Config becomes a line in the configuration file. e.g. SystemConfig .path = c:/users/app

Domain	Technical Name	Human Name	Documentation
Technology	dVolume	Volume	Refers to the storage assigned to containers, ensuring persistence, aligning with the Data Object in ArchiMate's Application Layer.
Technology	dZone	Zone	Defines macro divisions containing regions mapped to ArchiMate's Infrastructure Service.
Value types and commercial	dContract	Contract	Provides an agreement between a service consumer and a service provider that establishes functional and non-functional parameters for interaction.
Value types and commercial	dCostPeriod	Cost Period	The spend of an initiative in one period, planned or actual. A value type: it has no identity of its own.
Value types and commercial	dQuantityValue	Quantity	Represents a measurable value expressed as a numeric amount and a unit of measurement. It provides a reusable structure for quantitative properties of DAF elements.

7.2 DAF Relationship Types

Technical name	Human Name	Source [mult.] → Target [mult.]	Forward role / Backward role
ContractRelatedProduct	Contract is Related to Product	dContract [0..*] → dProduct [0..*]	relates to Product / is related to Contract
dAccord	According Opinion	dOpinion [1] → dOpinion [0..*]	is in accord with other Opinion / is in accord with original Opinion
dActionKey	Action Key	dController [0..*] → dController [0..*]	target Controller / Source Controller
dActivityContainsAction	Activity contains Action	dActivity [0..*] → dAction [0..*]	is contained in Activity / contains Action
dActorParticipatesInBizUseCase	Actor Participates In Use Case	dActor [0..*] → dBusinessUseCase [0..*]	Partecipates in Use Case / is executed by Actor
dAPIAggregatesSchema	Aggregates Schema	dAPI [0..*] → dJSON_Schema [0..*]	Aggregates Schema / is aggregates in API
dApplicationAggregatesFunction	aggregates Function	dApplicationComponent [0..1] → dApplicationFunction [0..*]	aggregates Function / part of Application
dApplicationComponentHasContainer	Application Has Container	dApplicationComponent [0..*] → dContainer [0..*]	Has Container / contains Application
dApplicationComponentRealizesLogicalComponent	Realizes Component	dApplicationComponent [0..*] → dLogicalAppComponent [0..1]	Realizes Logical component / is Realized by Component

Technical name	Human Name	Source [mult.] → Target [mult.]	Forward role / Backward role
dApplicationconfiguredbySystemConfiguration	Application configured by System Configuration	dApplicationComponent [0..*] → dSystem [0..1]	has configuration / configures Application
dApplicationConnectedWithApplication	Connects Application	dApplicationComponent [0..*] → dApplicationComponent [0..*]	connected From Application / Connected to Application
dApplicationExposesService	Application Exposes Ph. Service	dApplicationComponent [0..*] → dPhysicalService [0..*]	is exposed by Application / Exposes Phy Service
dApplicationHasOwner	Application has Owner	dApplicationComponent [0..*] → dStakeholder [0..*]	owns Application / has Owner
dApplicationRisk	Application Risk	dRisk [0..*] → dApplicationComponent [0..*]	is related to Application / has Risk
dApplicationRunsInNode	Application is run By Node	dDeploymentNode [0..*] → dApplicationComponent [0..*]	Runs Application / is run By Node
dAreaAggregatesCategories	Area aggregates Categories	dMeasurementArea [1] → dMeasurementCategory [0..*]	aggregates categories / has Measurement Area
dBizServIsDescribedByUseCase	Biz Serv Is Described By Use Case	dBusinessUseCase [0..*] → dBusinessService [0..*]	Has service / described by Use Case
dBizServUsesDataEntity	Service Uses Data	dBusinessService [0..*] → dDataEntity [0..*]	uses Data / is used by service
dBusinessAreaHasCapability	has Capability	dBusinessArea [1] → dCapability [1..*]	has Capability / is organized in Biz Area
dCapabilityisMeasuredByKPI	is Measured By KPI	dCapability [1..*] →	is Measured By Indicator / is measuring

Technical name	Human Name	Source [mult.] → Target [mult.]	Forward role / Backward role
		dMeasurementIndicator [0..*]	Capability
dCategoryaggregatesGroups	aggregates Groups	dMeasurementCategory [1] → dMeasurementGrouping [0..*]	aggregates Groups / has Category
dClusterContainsDeploymentNode	Cluster Contains Node	dCluster [1] → dDeploymentNode [0..*]	Contains DeploymentNode / is Contained in Cluster
dContainerAggregatesVolume	Container Aggregates Volume	dContainer [0..*] → dVolume [0..*]	Aggregates Volume / is Aggregated in Container
dContractServesActor	Contract Serves Actor	dContract [0..*] → dActor [0..*]	is serving actor / served by contract
dControllerControlsModel	Controls Model	dController [0..1] → dModelClass [0..*]	Controls Model / is controlled by Controller
dControllerGovernsView	Governs View	dController [0..1] → dView [0..*]	Governs view / is Governed by Controller
dControllerImplementsFunction	controller Implements function	dController [0..*] → dApplicationFunction [0..1]	Implements function / has Controller
dDataEntityInformsCapability	informs Capability	dCapability [0..*] → dDataEntity [0..*]	is informed by Data Entity / Informs Capability
dDecisionRefersToPrinciple	Decision refers To Principle	dDecision [0..*] → dPrinciple [1..*]	refers To Principle / is referenced by Decision
dDeploymentModelAggregatesCluster	Deployment Model Aggregates Cluster	dDeploymentModel [1] → dCluster [1..*]	Aggregates Cluster / is Aggregated in Deployment Model

Technical name	Human Name	Source [mult.] → Target [mult.]	Forward role / Backward role
dDiscord	Discording opinion	dOpinion [1] → dOpinion [0..*]	is in discord with other Opinion / is in discord with Original Opinion
dEntityAssociatesEntity	Entity Associates Entity	dDataEntity [0..*] → dDataEntity [0..*]	Is connected to Source Entity / Is connected to Target Entity
dEventLeadsToEvent	Event Leads to Event	dEvent [0..*] → dEvent [0..*]	leads to Event / follows Event
dEventTriggersProcess	Event Triggers Process	dEvent [0..*] → dBusinessProcess [0..*]	triggers Process / is triggered by Event
dFeatureIsRealizedByUseCase	Feature is realized by Use Case	dBusinessUseCase [0..*] → dFeature [0..*]	realizes Feature / is realized by Use Case
dGoalAggregatesGoal	Goal Aggregates Goal	dGoal [0..*] → dGoal [0..*]	parent Goal / Child Goal
dGoalhasObjective	Goal has Milestone	dGoal [0..*] → dObjective [0..*]	Has Objective / is related to Goal
dGoalisOperationalizedByCapability	goal is Operationalized By Capability	dCapability [0..*] → dGoal [1..*]	support Goal / is Operationalized By Capability
dGoalTargetsdMeasurementIndicator	Goal Targets KPI	dGoal [0..*] → dMeasurementIndicator [0..*]	Targets KPI / has target Goal
dGroupAggregatesKPI	Group aggregates KPI	dMeasurementGrouping [1..*] → dMeasurementIndicator [0..*]	aggregates KPI / part of a Grouping
dInitiativeIncreasesMaturityOf	Initiative Increases Maturity Of	dInitiative [0..*] → dCapability [0..*]	Increases Maturity Of Capability / is

Technical name	Human Name	Source [mult.] → Target [mult.]	Forward role / Backward role
	Capability		matured by Initiative
dInitiativeisMakingUseOfDecision	Initiative is Making Use Of Decision	dDecision [0..*] → dInitiative [0..*]	is used in Initiative / Makes Use Of Decision
dIssueStopsRealizationOfRequirement	Issue Stops Realization Of Req	dIssue [0..*] → dRequirement [0..*]	stop Requirement / is stopped by Issue
dJSON_SchemaSubSetGeneralizes_JSON_Schema	Schema Generalizes Schema	dJSON_Schema [0..*] → dJSON_SchemaSubSet [0..*]	Parent Schema / Child Schema
dJSONElement_associates_JSONElement	dJSONElement_associates_JSONElement	dJSON_Element [0..*] → dJSON_Element [0..*]	target JsonElement / Source JsonElement
dJSONSchemaRealizesModel	Schema Realizes Model	dJSON_Schema [0..*] → dModelClass [1]	realizes Model Class / is realized by Schema
dLocationHostsNode	Location Hosts Node	dLocation [1] → dDeploymentNode [0..*]	Hosts Node / Is hosted in Location
dLogAppCompExposesBizService	exposes Biz Service	dLogicalAppComponent [0..*] → dBusinessService [0..*]	application runs service / service run by application
dLogicalApplicationComponentSupportsCapability	App Supports Capability	dCapability [1..*] → dLogicalAppComponent [0..*]	supported by Application / supports Capability
dMeasurementIndicatorImprovesMeasurementIndicator	KPI Improves KPI	dMeasurementIndicator [0..*] → dMeasurementIndicator [0..*]	Improves KPI / Improved KPI
dMeasurementIndicatorWorsenMeasurementIndicator	Worsen KPI	dMeasurementIndicator [0..*] → dMeasurementIndicator [0..*]	Worsen KPI / is worsen by

Technical name	Human Name	Source [mult.] → Target [mult.]	Forward role / Backward role
dModelClassAssociatesModelClass	Model Associates Model	dModelClass [0..*] → dModelClass [0..*]	Associates Target Model / Associates Source Model
dModelClassRealizesEntity	Model Class Realizes Entity	dModelClass [1] → dDataEntity [0..1]	Realizes Entity / is realized by Class
dNetworkAggregatesSubNetwork	Network Aggregates SubNetwork	dNetwork [0..*] → dSubNetwork [0..*]	Aggregates Sub-Network / is Aggregated in Network
dNetworkContainsNode	Contains Node	dNetwork [0..1] → dDeploymentNode [0..*]	Contains Node / is contained in a Network
dNetworkHasSecurityGroup	Network Has Security Group	dNetwork [0..*] → dSecurityGroup [0..*]	Has Security Group / is associated to Network
dObjectInstanceofController	Instance of Controller	dObject [0..*] → dController [0..*]	Has an Instance / Istance of Controller
dObjectInstanceofModel	Instance of Model	dObject [0..*] → dModelClass [0..*]	Has an Instance / Istance of Model
dObjectInstanceofView	Instance of View	dObject [0..*] → dView [0..*]	Has an Instance / Istance of View
dOrgUnitHasStakeholders	Org Unit has Stakeholders	dOrganizationUnit [0..*] → dStakeholder [0..*]	has Stakeholder / is concerned with Organization
dPackageIsDeliveredByInitiative	Package is delivered by Initiative	dGrowthPackage [1] → dInitiative [0..*]	is delivered by Initiative / is part of a Package
dPhysicalServiceImplementsBusinessService	Physical Serv. Implements Biz Serv.	dPhysicalService [0..*] →	Implement Biz Service / is

Technical name	Human Name	Source [mult.] → Target [mult.]	Forward role / Backward role
		dBusinessService [0..*]	Implemented by physical service
dPolymorphicCollection	Polymorphic Collection	dJSON_Schema [0..*] → dJSON_Schema [0..*]	@baseType / @Type
dProcessEnsureCorrectOperationOfCapability	Ensure the correct Operation of	dCapability [0..*] → dBusinessProcess [0..*]	is operated by Process / Ensure the correct Operation of Capability
dProcessFlowsToProcess	Process Flows to Process	dBusinessProcess [0..*] → dBusinessProcess [0..*]	Sequence Flow To / Sequence Flow From
dProcessOrchestratesService	Process Orchestrates Service	dBusinessProcess [0..*] → dBusinessService [0..*]	Has Service / part of Process
dProcessResultsInEvent	Process Results in Event	dBusinessProcess [0..*] → dEvent [0..*]	results in Event / is result of Process
dProductIncludesProcess	Product Includes Process	dProduct [0..*] → dBusinessProcess [0..*]	Includes Process / Is part of Product
dProgramIsOrganizedInPackages	Program is organized In Packages	dProgram [1] → dGrowthPackage [0..*]	is organized In Packages / part of a Program
dRegionAggregatesCluster	Region Aggregates Cluster	dRegion [1] → dCluster [0..*]	Aggregates Cluster / is Aggregated in Region
dRelatedService	Child Process	dBusinessService [0..*] → dBusinessService [0..*]	Target Service / Source Service
dRequirementContainsRequirement	Requirement Contains Req	dRequirement [0..1] → dRequirement [0..*]	is contained by Requirement / Contains Requirement

Technical name	Human Name	Source [mult.] → Target [mult.]	Forward role / Backward role
dRequirementIsRelatedToGoal	req related to Goal	dRequirement [0..*] → dGoal [0..*]	specifies Goal / is specified by Requirement
dRequirementIsSatisfiedByFeature	is Satisfied By	dFeature [0..*] → dRequirement [0..*]	realizes Requirement / is realized by Feature
dResourceimpactedByRisk	Resource impacted by Risk	dRisk [0..*] → dResource [0..*]	impact Resouce / is impacted by Risk
dResourceSupportsCapability	Resource supports Capability	dResource [1..*] → dCapability [1..*]	Supports Capability / is supported By Resource
dRiskIsRelatedToRisk	Risk is related to Risk	dRisk [0..*] → dRisk [0..*]	target Risk / Source Risk
dRiskRealizedByIssue	realized by Issue	dIssue [0..1] → dRisk [0..1]	is realizing Risk / is realized by Issue
dRoleConsumesService	Role Consumes Service	dRole [0..*] → dBusinessService [0..*]	Consumes Service / is consumed by Role
dRoleExecutesCapability	role Executes Capability	dCapability [0..*] → dRole [0..*]	is executed by Role / executes Capability
dRoleHasSkill	Role has skill	dRole [0..*] → dSkill [0..*]	has SKill / known by Role
dRoleisAssociatedToOrganization	Role is Associated To Organization	dRole [0..*] → dOrganizationUnit [0..1]	works in Organization / has Role

Technical name	Human Name	Source [mult.] → Target [mult.]	Forward role / Backward role
dRoleIsPerformedByActor	Role isPerformed By Actor	dActor [0..*] → dRole [0..*]	has Role / is executed by Actor
dSchema_Associates_Schema	Schema Association	dJSON_Schema [0..*] → dJSON_Schema [0..*]	Target Schema / Source Schema
dSchemaAssociatesElement	Schema Associates Element	dJSON_Schema [0..*] → dJSON_Element [0..*]	Associates Element / Associates Schema
dSchemaGeneralizesSchema	Schema Generalization	dJSON_Schema [0..*] → dJSON_Schema [0..*]	Child Schema / Parent Schema
dSecurityGroupHasPhysicalService	Security Group Has Service	dSecurityGroup [0..*] → dPhysicalService [0..*]	has Physical Service / Asociated to Sec. group
dServiceRealizedByAPI	Service Realized By API	dAPI [0..*] → dPhysicalService [0..1]	realized Physical Service / is realized By API
dStakeholderHasInnerOpinion	Stakeholder Has Inner Opinion	dStakeholder [1] → dOpinionInner [0..*]	Has Inner Opinion / is thinked by Stakeholder
dStakeholderHasOpinion	Stakeholder Has Opinion	dStakeholder [1] → dOpinion [0..*]	Has Opinion / is expressed by Stakeholder
dSubNetworkAggregatesIPRange	SubNetwork Aggregates IP Range	dSubNetwork [0..*] → dIPRange [0..*]	Aggregates IP Range / is Aggregated in sub Net.
dTableProvidesPersistenceForModelClass	Table Provides Persistence to Model	dModelClass [1] → dTable [0..*]	is Saved in a Table / Provides Persistence For Model
dTest_ApplicationComponent	Test validates Component	dTest [0..*] → dApplicationComponent	Test validates Component / is validated

Technical name	Human Name	Source [mult.] → Target [mult.]	Forward role / Backward role
		[0..*]	by test
dUseCaseContainsActivities	Use Case Contains Activities	dBusinessUseCase [0..*] → dActivity [0..*]	has Activitiy / part of UseCase
dUseCaseHasStory	Use Case Has Story	dUserStory [0..*] → dBusinessUseCase [0..*]	Is associated to Use Case / Can be told as User Story
dUseCaseisImplementedByController	Use Case is Implemented By Controller	dController [0..*] → dBusinessUseCase [0..*]	implements Use Case / Is implemented by Controller
dValueStreamEnablesCapability	Value Stream Enables Capability	dValueStream [0..*] → dCapability [0..*]	Enables Capability / is Enabled by Value Stream
dValueStreamIncludesProcesses	Value Stream Includes Processes	dValueStream [0..*] → dBusinessProcess [0..*]	Includes Process / is part of Value Stream
dZoneAggregatesRegion	Zone Aggregates Region	dZone [1] → dRegion [0..*]	Aggregates Region / is Aggregated in Zone

7.2.1 DAF Relationships Quick link

Source → Relationship → Target	Forward role / backward role
`dActivity → dActivityContainsAction → dAction`	`is contained in Activity` `contains Action`
`dActor → dActorParticipatesInBizUseCase → dBusinessUseCase`	`Partecipates in Use Case` `is executed by Actor`
`dActor → dRoleIsPerformedByActor → dRole`	`has Role` `is executed by Actor`

Source → Relationship → Target	Forward role / backward role
dAPI → dAPIAggregatesSchema → dJSON_Schema	Aggregates Schema is aggregates in API
dAPI → dServiceRealizedByAPI → dPhysicalService	realized Physical Service is realized By API
dApplicationComponent → dApplicationConnectedWithApplication → dApplicationComponent	connected From Application Connected to Application
dApplicationComponent → dApplicationAggregatesFunction → dApplicationFunction	aggregates Function part of Application
dApplicationComponent → dApplicationComponentHasContainer → dContainer	Has Container contains Application
dApplicationComponent → dApplicationComponentRealizesLogicalComponent → dLogicalAppComponent	Realizes Logical component is Realized by Component
dApplicationComponent → dApplicationExposesService → dPhysicalService	is exposed by Application Exposes Phy Service
dApplicationComponent → dApplicationHasOwner → dStakeholder	owns Application has Owner
dApplicationComponent → dApplicationconfiguredbySystemConfiguration → dSystem	has configuration configures Application
dBusinessArea → dBusinessAreaHasCapability → dCapability	has Capability is organized in Biz Area
dBusinessProcess → dProcessFlowsToProcess → dBusinessProcess	Sequence Flow To Sequence Flow From
dBusinessProcess → dProcessOrchestratesService → dBusinessService	Has Service part of Process
dBusinessProcess → dProcessResultsInEvent → dEvent	results in Event is result of Process
dBusinessService → dRelatedService → dBusinessService	Target Service Source Service
dBusinessService → dBizServUsesDataEntity → dDataEntity	uses Data

Source → Relationship → Target	Forward role / backward role
	is used by service
dBusinessUseCase → dUseCaseContainsActivities → dActivity	has Activitiy part of UseCase
dBusinessUseCase → dBizServIsDescribedByUseCase → dBusinessService	Has service described by Use Case
dBusinessUseCase → dFeatureIsRealizedByUseCase → dFeature	realizes Feature is realized by Use Case
dCapability → dProcessEnsureCorrectOperationOfCapability → dBusinessProcess	is operated by Process Ensure the correct Operation of Capability
dCapability → dDataEntityInformsCapability → dDataEntity	is informed by Data Entity Informs Capability
dCapability → dGoalisOperationalizedByCapability → dGoal	support Goal is Operationalized By Capability
dCapability → dLogicalApplicationComponentSupportsCapability → dLogicalAppComponent	supported by Application supports Capability
dCapability → dCapabilityisMeasuredByKPI → dMeasurementIndicator	is Measured By Indicator is measuring Capability
dCapability → dRoleExecutesCapability → dRole	is executed by Role executes Capability
dCluster → dClusterContainsDeploymentNode → dDeploymentNode	Contains DeploymentNode is Contained in Cluster
dContainer → dContainerAggregatesVolume → dVolume	Aggregates Volume is Aggregated in Container
dContract → dContractServesActor → dActor	is serving actor served by contract
dContract → ContractRelatedProduct → dProduct	relates to Product is related to Contract
dController → dControllerImplementsFunction → dApplicationFunction	Implements function has Controller

Source → Relationship → Target	Forward role / backward role
dController → dUseCaseisImplementedByController → dBusinessUseCase	implements Use Case Is implemented by Controller
dController → dActionKey → dController	target Controller Source Controller
dController → dControllerControlsModel → dModelClass	Controls Model is controlled by Controller
dController → dControllerGovernsView → dView	Governs view is Governed by Controller
dDataEntity → dEntityAssociatesEntity → dDataEntity	Is connected to Source Entity Is connected to Target Entity
dDecision → dInitiativeisMakingUseOfDecision → dInitiative	is used in Initiative Makes Use Of Decision
dDecision → dDecisionRefersToPrinciple → dPrinciple	refers To Principle is referenced by Decision
dDeploymentModel → dDeploymentModelAggregatesCluster → dCluster	Aggregates Cluster is Aggregated in Deployment Model
dDeploymentNode → dApplicationRunsInNode → dApplicationComponent	Runs Application is run By Node
dEvent → dEventTriggersProcess → dBusinessProcess	triggers Process is triggered by Event
dEvent → dEventLeadsToEvent → dEvent	leads to Event follows Event
dFeature → dRequirementIsSatisfiedByFeature → dRequirement	realizes Requirement is realized by Feature
dGoal → dGoalAggregatesGoal → dGoal	parent Goal Child Goal
dGoal → dGoalTargetsdMeasurementIndicator → dMeasurementIndicator	Targets KPI has target Goal
dGoal → dGoalhasObjective → dObjective	Has Objective

Source → Relationship → Target	Forward role / backward role
	is related to Goal
dGrowthPackage → dPackageIsDeliveredByInitiative → dInitiative	is delivered by Initiative is part of a Package
dInitiative → dInitiativeIncreasesMaturityOf → dCapability	Increases Maturity Of Capability is matured by Initiative
dIssue → dIssueStopsRealizationOfRequirement → dRequirement	stop Requirement is stopped by Issue
dIssue → dRiskRealizedByIssue → dRisk	is realizing Risk is realized by Issue
dJSON_Element → dJSONElement_associates_JSONElement → dJSON_Element	target JsonElement Source JsonElement
dJSON_Schema → dSchemaAssociatesElement → dJSON_Element	Associates Element Associates Schema
dJSON_Schema → dPolymorphicCollection → dJSON_Schema	@baseType @Type
dJSON_Schema → dSchema_Associates_Schema → dJSON_Schema	Target Schema Source Schema
dJSON_Schema → dSchemaGeneralizesSchema → dJSON_Schema	Child Schema Parent Schema
dJSON_Schema → dJSON_SchemaSubSetGeneralizes_JSON_Schema → dJSON_SchemaSubSet	Parent Schema Child Schema
dJSON_Schema → dJSONSchemaRealizesModel → dModelClass	realizes Model Class is realized by Schema
dLocation → dLocationHostsNode → dDeploymentNode	Hosts Node Is hosted in Location
dLogicalAppComponent → dLogAppCompExposesBizService → dBusinessService	application runs service service run by application
dMeasurementArea → dAreaAggregatesCategories → dMeasurementCategory	aggregates categories has Measurement Area

Source → Relationship → Target	Forward role / backward role
dMeasurementCategory → dCategoryaggregatesGroups → dMeasurementGrouping	aggregates Groups has Category
dMeasurementGrouping → dGroupAggregatesKPI → dMeasurementIndicator	aggregates KPI part of a Grouping
dMeasurementIndicator → dMeasurementIndicatorImprovesMeasurementIndicator → dMeasurementIndicator	Improves KPI Improved KPI
dMeasurementIndicator → dMeasurementIndicatorWorsenMeasurementIndicator → dMeasurementIndicator	Worsen KPI is worsen by
dModelClass → dModelClassRealizesEntity → dDataEntity	Realizes Entity is realized by Class
dModelClass → dModelClassAssociatesModelClass → dModelClass	Associates Target Model Associates Source Model
dModelClass → dTableProvidesPersistenceForModelClass → dTable	is Saved in a Table Provides Persistence For Model
dNetwork → dNetworkContainsNode → dDeploymentNode	Contains Node is contained in a Network
dNetwork → dNetworkHasSecurityGroup → dSecurityGroup	Has Security Group is associated to Network
dNetwork → dNetworkAggregatesSubNetwork → dSubNetwork	Aggregates Sub-Network is Aggregated in Network
dObject → dObjectInstanceofController → dController	Has an Instance Istance of Controller
dObject → dObjectInstanceofModel → dModelClass	Has an Instance Istance of Model
dObject → dObjectInstanceofView → dView	Has an Instance Istance of View
dOpinion → dAccord → dOpinion	is in accord with other Opinion is in accord with original Opinion
dOpinion → dDiscord → dOpinion	is in discord with other Opinion

Source → Relationship → Target	Forward role / backward role
	is in discord with Original Opinion
dOrganizationUnit → dOrgUnitHasStakeholders → dStakeholder	has Stakeholder is concerned with Organization
dPhysicalService → dPhysicalServiceImplementsBusinessService → dBusinessService	Implement Biz Service is Implemented by physical service
dProduct → dProductIncludesProcess → dBusinessProcess	Includes Process Is part of Product
dProgram → dProgramIsOrganizedInPackages → dGrowthPackage	is organized In Packages part of a Program
dRegion → dRegionAggregatesCluster → dCluster	Aggregates Cluster is Aggregated in Region
dRequirement → dRequirementIsRelatedToGoal → dGoal	specifies Goal is specified by Requirement
dRequirement → dRequirementContainsRequirement → dRequirement	is contained by Requirement Contains Requirement
dResource → dResourceSupportsCapability → dCapability	Supports Capability is supported By Resource
dRisk → dApplicationRisk → dApplicationComponent	is related to Application has Risk
dRisk → dResourceimpactedByRisk → dResource	impact Resouce is impacted by Risk
dRisk → dRiskIsRelatedToRisk → dRisk	target Risk Source Risk
dRole → dRoleConsumesService → dBusinessService	Consumes Service is consumed by Role
dRole → dRoleisAssociatedToOrganization → dOrganizationUnit	works in Organization has Role
dRole → dRoleHasSkill → dSkill	has SKill known by Role

Source → Relationship → Target	Forward role / backward role
dSecurityGroup → dSecurityGroupHasPhysicalService → dPhysicalService	has Physical Service Asociated to Sec. group
dStakeholder → dStakeholderHasOpinion → dOpinion	Has Opinion is expressed by Stakeholder
dStakeholder → dStakeholderHasInnerOpinion → dOpinionInner	Has Inner Opinion is thinked by Stakeholder
dSubNetwork → dSubNetworkAggregatesIPRange → dIPRange	Aggregates IP Range is Aggregated in sub Net.
dTest → dTest_ApplicationComponent → dApplicationComponent	Test validates Component is validated by test
dUserStory → dUseCaseHasStory → dBusinessUseCase	Is associated to Use Case Can be told as User Story
dValueStream → dValueStreamIncludesProcesses → dBusinessProcess	Includes Process is part of Value Stream
dValueStream → dValueStreamEnablesCapability → dCapability	Enables Capability is Enabled by Value Stream
dZone → dZoneAggregatesRegion → dRegion	Aggregates Region is Aggregated in Zone

7.3 Visual Characteristics of some DAF stereotypes

Some DAF stereotypes have special visual characteristics. They can react to Tagged Values or other properties, e.g. diagram properties as shown below.

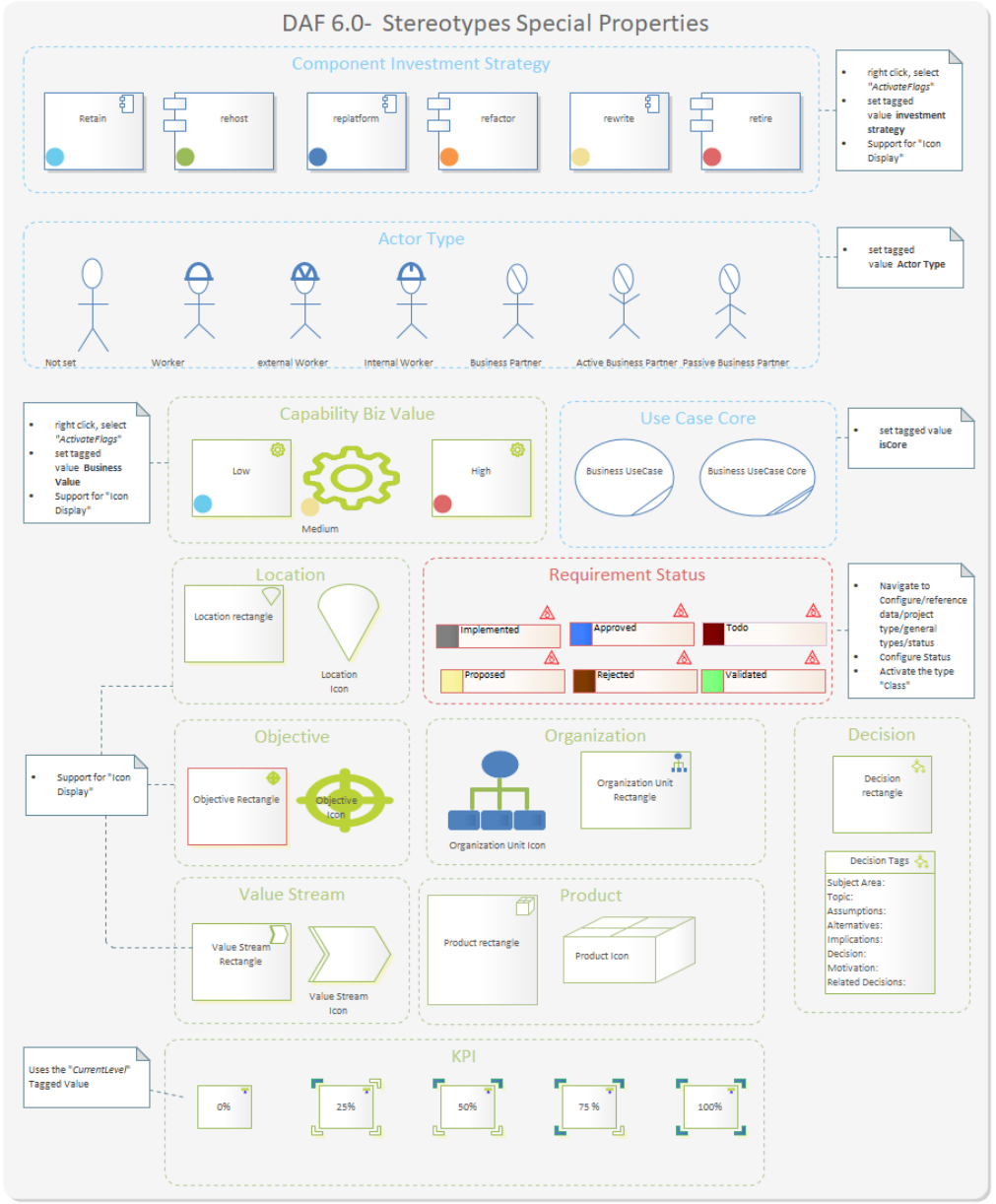


Figure 18: DAF Visual Properties

8 Appendices

8.1 Stereotypes and Tagged Values

Stereotype	Tagged Value : Type [multiplicity]	Notes
dAction		A specialized activity.
dActivity		Represents business or system activities. A function or work carried out by the business user.
dActor		Represents an entity (such as a person, system, or organization) that interacts with the system to achieve a specific goal. An actor typically performs actions within business Use cases and can be internal (e.g., employees) or external (e.g., customers or partner organizations), active and passive.
	ActorGoal : String [1]	Objectives that this actor has, in general terms.
	ActorTasks : String [1]	Tasks that this actor performs, in general terms.
	ActorType : ActorTypes [1]	the type of Actor
	Count : dQuantityValue [0..1]	Estimated number of FTEs that operate as this Actor.
dAPI		OpenAPI Specification is an open-source format and initiative for designing and creating machine-readable interface files that are utilized in producing, describing, consuming, and visualizing RESTful APIs and web services.
	basePath : String [1]	The base path on which the API is served, which is relative to the host. If it is not included, the API is served directly under the host. The value MUST start with a leading slash (/
	consumes : MimeTypes [0..*]	A list of MIME types the APIs can consume. This is global to all APIs but can be overridden on specific API calls.
	contact : Contact [1]	
	definitions : String [1]	An object to hold data types produced and consumed by operations.
	Full Name : String [1]	
	host : String [0..1]	The host (name or ip) serving the API. This MUST be the host only and does not include the scheme nor sub-paths. It MAY include a port. If the host is not included, the host serving the documentation is to be used (including the port). The host does not support path templating.
	license : String [1]	

Stereotype	Tagged Value : Type [multiplicity]	Notes
	parameters : String [1]	
	produces : MimeTypes [0..*]	
	responses : String [1]	
	schemes : SchemeType [0..*]	The transfer protocol of the API.
	securityDefinitions : String [1]	
	swagger : String [1]	OpenApi version Specifies the Swagger Specification version being used. It can be used by the Swagger UI and other clients to interpret the API listing. The value MUST be "2.0".
	tags : String [1]	
dAPIOperation		Defines operations exposed by an API for interaction with external systems. Also called endpoints.
	base_Operation : String [1]	
	consumes : String [1]	
	deprecated : String [1]	
	description : String [1]	
	hTTPMethod : HTTPMethod [1]	
	produces : String [1]	
	relativePath : String [1]	
	schemes : String [1]	
	summary : String [1]	
	tags : String [1]	
dAPIParameter		Defines parameters in API operations.

Stereotype	Tagged Value : Type [multiplicity]	Notes
	allowEmptyValue : String [1]	
	base_Parameter : String [1]	
	collectionFormat : String [1]	
	location : String [1]	
	required : String [1]	
dAPIResponse		Represents responses returned by API operations.
	base_Parameter : String [1]	
	default : String [1]	
	examples : String [1]	
	headers : String [1]	
	Integer : Int32 [1]	
dApplicationComponent		An application component is a self-contained unit. As such, it is independently deployable, re-usable, and replaceable. An application component performs one or more application functions. It encapsulates its behavior and data, exposes services, and makes them available through interfaces. Cooperating application components are connected via application collaborations. An application component may be assigned to one or more application functions. An application component has one or more application interfaces, which expose its functionality. Application interfaces of other application components may serve an application component. The name of an application component should preferably be a noun.
	ApplicationType : dApplicationType [1]	the type of Application, e.g. if it's a COTS, Custom and so on
	BizCriticality : Int32 [1]	how is Business happy with this application, often related to business debt.
	BizSatisfaction : Int32 [1]	How much is the business happy with the application, expressed in 1-100
	Category : String [1]	User-definable categorization taxonomy for each element.
	Cost : dQuantityValue [0..1]	the cost of this application in 1000 of $

Stereotype	Tagged Value : Type [multiplicity]	Notes
	InvestmentStrategy : dApplicationStrategy [1]	
	ITSatisfaction : Int32 [1]	how is IT happy with this application, often related to technical debt expressed in 1-100
	LastStandardReviewDate : DateTime [1]	Last review date for the application component.
	NextStandardReviewDate : DateTime [1]	Next review date for the application component.
	Owner : String [1]	The owner for the application Component.
	Port : String [1]	
	RetireDate : Date [0..1]	The retire date for the Application Component.
	Source : String [1]	The original source of the element's information
dApplicationFunction		Encapsulates functions supported by applications.
dBusinessArea		an organization of Business interests, can be decomposed.
dBusinessProcess		A sequence of activities that produce a specific service or product supporting a particular business capability
	Category : String [1]	User-definable categorization.
	isAdopted : dboolean [1]	
	isDocumented : dboolean [1]	
	isEffective : dboolean [1]	
	isOptimized : dboolean [1]	
	LastStandardReviewDate : DateTime [1]	Last date that the standard was reviewed.
	NextStandardReviewDate : DateTime [1]	Next date for the standard to be reviewed.
	Owner : String [1]	Owner of the architecture entity.
	ProcessCriticality : ProcessCriticality [1]	Criticality of this process to business operations.

Stereotype	Tagged Value : Type [multiplicity]	Notes
	ProcessType : dProcessType [1]	Whether this process is supported by IT or is a manual process.
	ProcessVolumetrics : dQuantityValue [0..1]	Data on frequency of process execution.
	RetireDate : Date [0..1]	Date when the standard was/will be retired.
	Source : String [1]	Location from where the information was collected.
	StandardCreationDate : Date [1]	If the product is a standard, when the standard was created.
dBusinessService		A repeatable activity; a discrete behavior that a building block may be requested or otherwise triggered to perform. A service provided by Business that achieve a business outcome in response to a request.
	Category : String [1]	User-definable categorization for Business Service.
	LastStandardReviewDate : Date [1]	Last date that the standard was reviewed.
	NextStandardReviewDate : Date [1]	Next date for the standard to be reviewed.
	Owner : String [1]	Owner of the architecture entity.
	RetireDate : Date [0..1]	Date when the standard was/will be retired.
	Source : String [1]	Location from where the information was collected.
dBusinessUseCase		Encapsulates business processes producing value for an enterprise.
	Extensions : String [1]	
	GoalInContext : String [1]	The goal should implicitly express the actor's intent or purpose of the use case, such as "Enrol Student in Seminar."
	isCore : Boolean [1]	
	Level : String [1]	Authorizations for operations/actions to be performed against the Chi business objects in scope. Against every object/process 4 CRUD basic operations are possible: Create (Write) Read (Open) Update (Change)

Stereotype	Tagged Value : Type [multiplicity]	Notes
		Delete (Destroy)
	MainSuccessScenario : String [1]	The main path of logic an actor follows through a use case. Often referred to as the "happy path" or the "main path" because it describes how the use case works when everything works as it normally should.
	OtherActors : String [1]	The list of actors associated with the use case. Although this information is contained in the use case itself, it helps to increase the understandability of the use case when the diagram is unavailable.
	Precondition : String [1]	A list of the conditions, if any, that must be met before a use case may be invoked. Can be a previous Use case or self the presence of the system in Scope.
	Scope : String [1]	Boundaries in which the use case is operated when invoked (E.g. CMS)
	Trigger : String [1]	Event that is responsible for invocation of the use case.
dCapability		Represents an organizational capability to achieve business outcomes.Can be current available in an Organization Unit (or) A business-focused outcome that is delivered by the completion of one or more work packages (by increasing the capability Maturity).
	BusinessValue : CapabilityBusinessValue [1]	assess how this capability provides value to the enterprise.
	CapabilityLevel : Int32 [0..1]	the level of this category in the hierarchy
	Category : String [1]	User-definable categorization for Capability
	Cost : dQuantityValue [0..1]	the aggregated cost of this Capability. Includes all the aggregated cost's elements
	Criticality : Int32 [1]	What is the criticality of this capability, what would be the cost of failure expressed in 1-100
	Increments : dIncrement [1]	Current (AS-IS) maturity level for the entity. 0- none 1- Initial 2- UnderDevelopment 3 - Defined 4- Managed 5 - Measured
	IncrementsToBe : dIncrement [1]	Future (To-Be) maturity level for the entity.
	IncrementSupplyChain : dIncrement [1]	The maturity level of your supply chain (Partners)

Stereotype	Tagged Value : Type [multiplicity]	Notes
	IncrementVertical : dIncrement [1]	The maturity level of your vertical (the industry)
	Owner : String [1]	Owner of the architecture entity.
	Risk : Int32 [1]	How much Risk connected to this business Capability. 1 low risk, 100 max risk.
	Source : String [1]	Source from where the information was collected.
dCluster		Represents a set of nodes working in the same Region, akin to ArchiMate's Node concept within a server infrastructure.
dContainer		Encapsulates application components and services, mirroring ArchiMate's concept of a Node, where an application runs.
dContract		Provides an agreement between a service consumer and a service provider that establishes functional and non-functional parameters for interaction.
	AvailabilityCharacteristics : String [0..1]	Degree to which something is available for use.
	billableUnit : String [0..1]	The unit that is billed, as an ISO 4217 or UCUM code, e.g. {transaction}, GBy.
	ConsumingService : String [0..1]	The service which consumes the contract.
	Description : String [0..1]	Description for the contract.
	effectiveFrom : Date [0..1]	The first day the contract applies.
	effectiveTo : Date [0..1]	The last day the contract applies; after effectiveFrom.
	Growth : String [0..1]	Expected future growth rate of service request.
	Owner : dRole [0..1]	Owner of the architecture entity.
	pricingBasis : String [0..1]	What the price is charged for, e.g. per subscriber, per transaction, flat fee.
	ProvidingService : String [0..1]	The service which provides this Contract.
	renewal : Duration [0..1]	The renewal term, e.g. P1Y.
	ResponseRequirements : String [0..1]	Response SLA times that the service provider must meet for particular operations.

Stereotype	Tagged Value : Type [multiplicity]	Notes
	ServiceTimes : String [0..1]	Hours during which the service must be available.
	Source : String [0..1]	Location from where the information was collected.
	tariff : dQuantityValue [0..1]	The price per billable unit, e.g. 0.02 EUR per {transaction}.
	Throughput : String [0..1]	Required throughput capacity.
	Invariant	effectiveTo is after effectiveFrom when both are set
dController		Encapsulates application behavior and flow logic.
dCostPeriod		The spend of an initiative in one period, planned or actual. A value type: it has no identity of its own.
	amount : dQuantityValue [1]	The money spent in the period; unit is an ISO 4217 code.
	costNature : dCostNature [1]	Capex or Opex.
	periodLength : Duration [1]	The length of the period as an xsd:duration, e.g. P1M, P3M.
	periodStart : Date [1]	The first day of the period.
	Invariant	amount is money: its unit is an ISO 4217 code
dDataEntity		An encapsulation of data that is recognized by a business domain expert as a thing. Logical data entities can be tied to applications, repositories, and services and may be structured according to implementation considerations.
	Category : String [1]	User-definable categorization.
	isAccessible : dboolean [1]	if this data entity is accessible or not
	isAccurate : boolean [1]	if this data entity represents a valid reppresentation of the original concept(s)
	isTimely : dboolean [1]	if this data entity was updated and present current information
	Owner : String [1]	Owner of the Data Entity.
	PrivacyClassification : String [1]	Level of restriction placed on access to the data.

Stereotype	Tagged Value : Type [multiplicity]	Notes
	RetentionClassification : String [1]	Level of retention to be placed on the data.
	Source : String [1]	Location from where the information was collected.
dDecision		Represents decisions and their impact on architecture or processes.
	Alternatives : String [1]	A list of alternatives and explanations
	Assumptions : String [1]	What is believed to be true about the context of the problem, constraints on the solution.
	Decision : String [1]	The decision taken, possibly with references to related work products
	Implications : String [1]	What impact the decision will have
	Justification : Memo [1]	Why the decision was made and a list of compliance to Architecture Principles and explanations of deviations from compliance.
	Motivation : String [1]	Why this decision is important.
	RelatedDecisions : dDecision [1..*]	A list of related decisions
	Subject Area : String [1]	Area of Concern
	Topic : String [1]	Topic of Interest
dDeploymentModel		The root class for the description of a Deployments to roll out an application, service, or container to a set of resources. akin to ArchiMate's Technology Layer deployment views.
dDeploymentNode		Represents a physical machine or virtual server that hosts services, applications, or containers, akin to ArchiMate’s Node concept in the Technology Layer.
	ArchitectureType : ArchitectureType [1]	the type of CPU Architecture (e.g. AMD 64)
	CPU : String [1]	the number and type of CPU
	Disk : String [1]	the dimension of the disk
	hostname : String [1]	
	IP : String [1]	The Ip of this machine

Stereotype	Tagged Value : Type [multiplicity]	Notes
	OS : String [1]	the operating system installed in this node
	Owner : String [1]	THe owner / responsible for this node
	PublicIP : String [1]	
	RAM : String [1]	the Memory dimension
dEvent		Something that happens, inside or outside the organization, that triggers behaviour or results from it.
	duration : Duration [0..1]	How long the event lasts.
	endDate : DateTime [0..1]	The end of that period; after startDate.
	eventType : dEventType [0..1]	Where the event comes from.
	frequency : dQuantityValue [0..1]	For recurring events, e.g. 120 {claim} per P1D.
	likelihood : Int32 [0..1]	For possible events; scale 1-100, as dRisk.likelihood.
	nature : dEventNature [0..1]	Recurring: a designed trigger. Possible: part of a scenario. Occurred: a recorded occurrence.
	owner : dRole [0..1]	The role accountable for responding to the event.
	Source : String [0..1]	Location from where the information was collected.
	startDate : DateTime [0..1]	When an occurred event started, or the start of the window of a possible one.
	Invariant	frequency is set only when nature is Recurring
	Invariant	likelihood is set only when nature is Possible
	Invariant	endDate is after startDate when both are set
dFeature		A Feature is a fact that set his related Requirement to be true
	Author : String [1]	This feature's author's name and role in the project
	isImplemented : dboolean [1]	This feature's status: it's implemented ?

Stereotype	Tagged Value : Type [multiplicity]	Notes
	Proofreader : String [1]	This feature's revisor's name and role in the project
dGoal		a Goal is a measurable scope that the organization wants to achieve. Can be hierarchically decomposed.
	AssumptionType : dAssumptionType [0..*]	The assumptions made to achieve the goal.
	Priority : Int32 [1]	A priority in %
	Value_amount : dQuantityValue [0..1]	The actual amount of the value this goal intends to alter
	Value_Goal : dQuantityValue [0..1]	The target value this goal intend to achieve
	Value_Name : String [1]	The name of the value this goal intends to alter
dGrowthPackage		A set of actions identified to achieve one or more objectives for the business. A work package can be a part of an Initiative, a complete project, or a part of a Program. A Growth pack is a collection of Initiatives architected for consistency and aimed to be executed in the same period.
dInitiative		An activity carried out by a Business Unit or organization to achieve a particular goal or improve the Maturity of a certain capability.
	ability to Implement : Int32 [1]	expressed in % 1-100
	costProfile : dCostPeriod [0..*]	Planned or actual spend per period, ordered by periodStart.
	Description : String [1]	Description for the initiative.
	DetailedDescription : String [1]	Detailed Description of the initiative.
	EstimatedTotalCost : dQuantityValue [0..1]	the current estimation of the cost in the current currency (default USD)
	finishDate : Date [1]	The finish date of the initiative.
	Impacted Capability : dCapability [0..*]	Describes the contribution this work package makes to capability delivery.
	InitiativeDuration : Duration [0..1]	The expected duration of the initiative in Months/ Weeks or Years.
	initiativeHealth : InitiativeStatus [1]	a set of colors to Visually describe the health of the initiative.
	kind : String [1]	To capture the type of initiative.

Stereotype	Tagged Value : Type [multiplicity]	Notes
	purpose : String [1]	Purpose of the initiative.
	Rank : String [1]	The rank of the initiative.
	RelatedProgram : String [0..1]	The Growth Package or Program associated to the Initiative.
	startDate : Date [1]	The start date for the initiative.
	Invariant	InitiativeDuration must equal finishDate - startDate when all three are set
	Invariant	costProfile periods do not overlap
	Invariant	the sum of costProfile amounts equals EstimatedTotalCost when both are set (warning)
dIPRange		Defines a range of IP addresses (CIDR) used within a network. This aligns with ArchiMate's concept of Infrastructure Service, providing connectivity.
	CIDR : String [1]	The term CIDR (Classless Inter-Domain Routing) range refers to an IP address range that is defined by a combination of an IP address and a routing prefix (or subnet mask). CIDR notation is used to specify IP address ranges in a compact format, which is widely used in network configuration, including the configuration of cloud resources, subnets, or containers. CIDR range is written as an IP address followed by a slash ("/") and a number. For example, 192.168.0.0/24 refers to an IP range where the first 24 bits are network bits, and the remaining bits (in this case, 8 bits) are used for defining host addresses. This notation helps specify networks of varying sizes and allows more flexible allocation of IP addresses compared to the traditional class-based method.
	ipRangeType : ipRangeType [1]	
	Public : dboolean [1]	
dIssue		Am issue is a concrete problem that is affecting a business. Also, a fact that sets a Requirement to be false. (see also Risk)
	Author : String [1]	This issue's author's name and role in the project
	Responsible : String [1]	The responsible to close the present issue
dJSON_Attribute		Defines attributes in JSON data structures.
	jSONtype : dJSON_Type [1]	

Stereotype	Tagged Value : Type [multiplicity]	Notes
dJSON_Datatype		Represents a data type used within JSON structures, defining the format and constraints of data elements in a schema. JSON Data Types include standard types such as string, number, object, array, boolean, and null, ensuring consistency and validation in data exchange between systems.
dJSON_Element		Represents elements or nodes in JSON schemas.
dJSON_Schema		The Schema Object allows the definition of input and output data types. These types can be objects, but also primitives and arrays. This object is based on the JSON Schema Specification Draft 4 and uses a predefined subset of it. On top of this subset, there are extensions provided by this specification to allow for more complete documentation
	schema : SchemaVersion [1]	
	schemaFileName : String [0..1]	
dJSON_SchemaSubSet		Represents subsets of JSON schema structures.
	default : String [0..1]	
	enum : string [0..1]	
	exclusivemaximum : String [0..1]	
	exclusiveminimum : String [0..1]	
	format : format [0..1]	
	maximum : String [0..1]	
	maxItems : String [0..1]	
	maxlength : String [0..1]	
	maxProperties : String [0..1]	
	minimum : String [0..1]	
	minItems : String [0..1]	
	minlength : String [0..1]	

Stereotype	Tagged Value : Type [multiplicity]	Notes
	minProperties : String [0..1]	
	multipleof : String [0..1]	
	pattern : String [0..1]	
	required : String [0..1]	
	uniqueItems : String [0..1]	
dJSON_Type		Represents data type definitions for structured information.
	enum : String [0..1]	
	exclusivemaximum : String [0..1]	
	exclusiveminimum : String [0..1]	
	format : format [0..1]	
	maximum : String [0..1]	
	maxlength : String [0..1]	
	minimum : Int32 [0..1]	
	minlength : Int32 [0..1]	
	multipleof : String [0..1]	
	pattern : String [0..1]	
dLocation		A place where business activity takes place and can be hierarchically decomposed.
	AreaCode : String [1]	The area code of the Location.
	City : String [1]	The City of the Location.
	Country : String [1]	The City of the Location.

Stereotype	Tagged Value : Type [multiplicity]	Notes
	EmailID : String [1]	The contact e-mail Id for the Location.
	PhoneNumber : String [1]	The contact phone number for the Location.
	Province : String [1]	The Province of the Location.
	Street : String [1]	The street part of the Location.
dLogicalAppComponent		Encapsulates logical software components independent of implementation or technology.
	isMeetingBusinessNeeds : dboolean [1]	
	isMeetingFutureNeeds : dboolean [1]	
	isUsable : dboolean [1]	
dMeasurementArea		Represents areas of organizational or business performance measurement. The organization or Business units performance objective is captured as measurement area.
	Definition : String [1]	Definition of the Measurement Area.
dMeasurementCategory		Categorizes measurements within a measurement area.
	Definition : String [1]	Definition of the measurement category.
	MeasurementArea : dMeasurementArea [1]	The reference Measurement Area to which this category belongs to.
dMeasurementGrouping		a groups of Measurement Indicator that can be aggregated into Categories
	Definition : String [1]	Definition of the Measurement Grouping.
	MeasurementCategory : dMeasurementCategory [1]	Related Measurement Category to which the group belongs to.
dMeasurementIndicator		The specific measure captured for a Organization or Business Unit which can be measured and quantifiable. A measurement is a Key Performance indicator
	CurrentLevel : dQuantityValue [0..1]	the level of the Measurment Indicator the last time that was measured.
	Definition : String [1]	Definition of the Measurement Indicator.

Stereotype	Tagged Value : Type [multiplicity]	Notes
	LastStandardReviewDate : DateTime [1]	last time this KPI was reviewed
	SatisfactionLevel : dQuantityValue [0..1]	Satisfaction level for the Measurement Indicator, target amount to be reached and/or Max value
	UnitOfMeasure : String [0..1]	The unit of measure (if available) for the Measurement Indicator. String, 0..1, ISO 4217 or UCUM code; CurrentLevel.unit must equal it ISO 4217 or UCUM code.
	Invariant	CurrentLevel.unit must equal UnitOfMeasure
dModelClass		Represent a class of the domain model, that can be converted into a Persistent domain class, must inherit from Node. It is the application's dynamic data structure
	_metatype : String [1]	
	child_order : String [1]	The order of the associated children e.g. for Recipe: Image|Info|AdminInfo
	display_value : String [1]	The display value of a node e.g. for Recipe: name|AdminInfo/status|Info/status
	initparams : String [1]	The configuration section that is used on initialization of the corresponding mapper.
	is_searchable : boolean [1]	Indicates wether this type should be included in the default search.
	is_soap : boolean [1]	Indicates wennever an interface (e.g SOAP or REST) should be generated for this type.
	orderby : String [1]	Definition of default sorting. Possible values: 'none' (no order), 'sortkey' (generates a 'sortkey' column, that is used for explicit sorting) or any the name of any dValue defined in the node optionally followed by [ASC|DESC] e.g. 'name ASC'
	parent_order : String [1]	The order of the associated parents.
	pk_name : String [1]	The name of the primary key column. The generator will add this if there is no appropriate attribute.
	table_name : String [1]	The name of the database table. If not given the name will be taken.
dNetwork		The main communication medium between services and deployments, mapped to ArchiMate Communication Networks, facilitating inter-service communication.
	NetworkQuality : String [1]	
	NetworkType : NetworkTypes [1]	

Stereotype	Tagged Value : Type [multiplicity]	Notes
dObject		an Object is an instance of a class in a particular moment in runtime that can have its own state and data values.
dObjective		The Objective is a non material achievement, other than a Goal, the objective has no specific value to be measured. it's a time-bounded milestone for an organization used to demonstrate progress towards a goal.
	Category : String [1]	User-definable categorization.
	finishDate : Date [1]	the date in which the objective is achieved.
	Owner : String [0..1]	Owner for the Objective.
	Source : String [1]	Location from where the information was collected.
dOnPremise		an organization of Business interests
dOpinion		A view or judgment formed about a Topic. That is formulated in an explicit fashion. Captures explicit stakeholder judgments on a topic.
dOpinionInner		Captures implicit stakeholder opinions related to a topic. A Stakeholder’s opinion that is relevant for the topic but is not formulated explicitly.
dOrganizationUnit		Defines organizational units responsible for roles, can be decomposed
	HeadCount : dQuantityValue [0..1]	Number of FTEs working within the organization. can be calculated by aggregating roles' FTE unit {person}
dPhysicalService		a technical implementation of a service that runs on physical or virtual infrastructure, providing specific functionalities, usually in the form of software applications, middleware, databases, or platforms. It is concerned with the deployment, operation, and management of the technology stack, ensuring that the service is available, performant, and reliable. Examples include web servers, APIs, databases, and network services.
	Category : String [1]	
	execution_mode : executionMode [1]	
	LastStandardReviewDate : Date [1]	
	NextStandardReviewDate : String [1]	
	Owner : String [1]	

Stereotype	Tagged Value : Type [multiplicity]	Notes
	port : String [1]	
	protocol : ProtocolTypes [1]	
	RetireDate : Date [0..1]	
	Source : String [1]	
dPrinciple		Defines qualitative statements guiding architecture intent.
	Implications : String [1]	Statement of what the principle means in practical terms.
	Owner : dStakeholder [1]	Owner for the Principle
	PrincipleID : String [1]	Unique identifier for the Principle expressed as text.
	PrincipleMeasurement : dMeasurementIndicator [0..1]	Identifies mechanisms that will be used to measure whether the principle has been met or not.
	Priority : String [1]	Priority of this principle relative to other principles.
	Rationale : String [1]	Statement of why the principle is required and the outcome to be reached.
	Source : String [1]	Location from where the information was collected.
	Statement : String [1]	Statement of what the principle is.
	Type : dPrincipleType [1]	The category of the principle.
dProduct		Encapsulates outputs produced by business processes.
	Owner : dRole [1]	Owner of the Product.
	Price : dQuantityValue [0..1]	
	RetireDate : Date [0..1]	
	Source : String [0..1]	Location from where the information was collected.
dProgram		A group of Growth Packages that achieve a complex outcome for the organization

Stereotype	Tagged Value : Type [multiplicity]	Notes
dPublicCluster		A public cloud-based cluster, following the ArchiMate concept of Infrastructure Service provided by external cloud providers (Google, AWS, etc.).
	configFile : String [1]	config_google.conf
	NodeNum : String [1]	number of node in the cluster
	provider : dCloudProvider [1]	
dQuantityValue		Represents a measurable value expressed as a numeric amount and a unit of measurement. It provides a reusable structure for quantitative properties of DAF elements.
	amount : Decimal [1]	use QUDT: an ontology with QuantityValue and unit concepts. Better aligned with DAF's RDF/semantic model.
	asOf : Date [0..1]	the date the value refers to
	basis : String [0..1]	denominator, e.g. per FTE, per claim
	kind : dQuantityKind [0..1]	
	per : Duration [0..1]	Must be a valid xsd:duration value. ; xsd:duration is the XML/RDF prefix notation. Import or define the corresponding datatype in your UML model, since it is not a built-in UML primitive. Values use P for the duration and T before the time portion: - PT30M = 30 minutes - PT24H = 24 hours - PT72H = 72 hours - P7D = 7 days - P1M = 1 month - P1Y = 1 year - P2DT3H30M = 2 days, 3 hours, 30 minutes
	unit : String [1]	ISO 4217 for money, otherwise a unit IRI
	Invariant	unit must be a valid ISO 4217 or UCUM code
dRegion		A geographic area containing clusters and zones, aligned with ArchiMate's Location concept for representing deployment geography.
dRequirement		A Business guide line about the Enterprise or the project. Requirements can be decomposed.

Stereotype	Tagged Value : Type [multiplicity]	Notes
		A requirement is formulated in a SMART fashion and uses Moscow verbs (Must Have, Should Have, Could Have, and Would Like to Have.). Must Have: These are the requirements that are considered essential for the success of the initiative and must be meet for the system to function. They are often referred to as "hard" or "mandatory". requirements and are typically expressed in Boolean format as "true" or "1". Should Have: These are the requirements that are considered important, but not essential. They are typically expressed in Boolean format as "maybe" or "0/1" and are often used to prioritize the development of the system. Could Have: These are the requirements that are considered desirable but not essential. They are typically expressed in Boolean format as "false" or "0". They are used to identify potential enhancements to the system that can be added at a later date. Would Like to Have: These are the requirements that are considered less important and are often used to identify potential future enhancements to the system. In addition, the requirement should be SMART, which stands for Specific, Measurable, Achievable, Relevant, and Time-bound. SMART requirements are clear, concise, and easy to understand, and they provide a clear way to measure the success of the project.
	Author : String [1]	This requirement's author's name and role in the project
	Priority : Int32 [1]	A priority in %. Requirements are ordered by priority.
	Proofreader : String [1]	Each requirement needs to be confirmed. This requirement's proofreader 's name and role in the project
	Status : String [1]	the status of the Requirement in the workflow.
	Type : dRequirementType [1]	The type of requirement.
dResource		a type of Asset supporting a Business capability
	# of items : Float [1]	
dRisk		a risk Identifies and assesses potential issue impacting other elements. It is a potential future Issue. Can be hierarchically decomposed.
	likelihood : Int32 [0..1]	the probability that this risk becomes an Issue. scale 1–100

Stereotype	Tagged Value : Type [multiplicity]	Notes
	RiskStrategy : dRiskStrategy [1]	the strategy to be adopted to manage the risk.
	Severity : String [1]	the expected impact if the risk materializes.
dRole		The usual or expected function of an actor, or the part somebody or something plays in a particular action or event. An actor may have a number of roles.
	#FTEs : dQuantityValue [0..1]	Number of Full time Equivalent employees working in the Role. unit {FTE}
	Cost : dQuantityValue [0..1]	the Cost of this Role
	DegreeOfUtilisation : String [1]	the Degree of utilization of this Role in %
	isSkillsAvailable : dboolean [1]	is the set of skill for this role available
	isSkillsDefined : dboolean [1]	is a set of precises skill for this role defined
	Owner : String [1]	Owner of the Role.
	Source : String [1]	Location from where the information was collected.
	User : String [1]	User-definable categorization.
dSecurityGroup		Controls and manages access and permissions between services, in line with TOGAF's Security Architecture for protecting digital assets.
dSkill		an ability that some role possesses with various degree
	SkillCategory : String [1]	
	SkillLevel : String [1]	
dStakeholder		Represents any party or force influencing the domain of interest. Any person or force that have an interest in the considered domain. This is typically indicated by name, can be connected with a Role in the organization. A stakeholder has open and Inner opinions that are relevant for the subject matter.
	Attitude : dScala [1]	
	Knowledge : dScala [1]	

Stereotype	Tagged Value : Type [multiplicity]	Notes
	Legitimacy : dScala [1]	Measure the degree of general acceptance of this stakeholder influence
	Power : dScala [1]	the measure of the possibilities in context that this person has to realize his objectives.
	RACI : String [1]	A RACI responsibility matrix defines roles and responsibilities across a project or process by categorizing tasks into four key areas. Here's a breakdown of each role: Responsible (R): The person or group performing the task or activity.Directly responsible for completing the work.They take action to accomplish the assigned task.There must be at least one responsible party per task. Accountable (A): The person or group ultimately answerable for the task’s completion. They delegate and oversee the work performed by the responsible party.Holds the authority to make final decisions.Only one individual is accountable for each task. Consulted (C): Subject matter experts who provide input before the task is completed. They are actively involved by giving advice and feedback. Two-way communication is needed between consulted and responsible parties. Informed (I): Individuals or groups that need to be kept updated on the progress or outcome.They are not directly involved in the task but need awareness of results.Typically, this is one-way communication.
	role : dRole [0..1]	
	Urgency : dScala [1]	the timely importance of the Stakeholder opinions
dSubNetwork		A subset of a network, used to organize and isolate portions of the network. Equivalent to an ArchiMate Node, responsible for networking resources.
dSystem		A system configuration. Settings are modeled as attributes with default values, each attribute of the System Config becomes a line in the configuration file. e.g. SystemConfig .path = c:/users/app
	config : String [1]	The configuration file where the settings will be placed.
	platform : String [1]	The plattform to which the configuration settings apply.

Stereotype	Tagged Value : Type [multiplicity]	Notes
dTable		Represents physical tables in a database schema.
	Database : String [1]	The DB schema hosting the table
dTest		Represents a mechanism for verifying or validating the functionality, behavior, or compliance of am application component . Tests can include automated or manual procedures designed to ensure alignment with requirements, detect defects, and confirm expected outcomes.
dUserStory		Captures user-centric requirements expressed as stories.
	EstimatedEffort : String [1]	Effort expressed in hours
	StoryDetails : Story [1]	intention of the story expressed as "As a Actor I want to do something So that I achieve Value"
	TestScenario : String [1]	The situation that verifies the user story is realized
dValue		An attribute of a Node value type used in DAF. These values are persistent
	app_data_type : String [1]	The datatype used in the application e.g. DATATYPE_DONTCARE, DATATYPE_ATTRIBUTE, DATATYPE_ELEMENT or DATATYPE_IGNORE
	column_name : String [1]	The name of the database column. If not given the name will be taken
	db_data_type : String [1]	The datatype used in the database
	display_type : String [1]	The HTML display type for the attribute e.g. image
	input_type : String [1]	The HTML input type for the attribute e.g. select#fix:key1[val1]|key2[val2]
	is_editable : boolean [1]	Whether the attribute is editable or not
	restrictions_description : String [1]	A text describing the restrictions
	restrictions_match : String [1]	A regular expression that the value must match (e.g. '[0-3][0-9]\.[0-1][0-9]\.[0-9][0-9][0-9][0-9]' for date values)
	restrictions_not_match : String [1]	A regular expression that the value must NOT match
dValueRef		A Node value type used in DAF that reference another Node Value, similar to a foreign Key. These values are persistent

Stereotype	Tagged Value : Type [multiplicity]	Notes
	reference_value : dValue [1]	
dValueStream		A representation of an end-to end collection of value-adding Business Processes that create an overall result for a customer, stakeholder or end user. A sequence of activities an enterprise undertakes to deliver on a customer request. More broadly, the sequence of activities required to design, produce, and deliver a good or service to a customer, and it includes the dual flows of information and material.
	Criticality : ValueStreamCriticality [0..1]	
	entrance criteria : String [0..1]	
	exit criteria : String [0..1]	
	is Decomposed : Boolean [1]	
dView		A View used in DAF. Controller, Views and Associations define the application flow
dVolume		Refers to the storage assigned to containers, ensuring persistence, aligning with the Data Object in ArchiMate’s Application Layer.
	MountPath : String [1]	
	name : String [1]	
	subPath : String [1]	
dZone		Defines macro divisions containing regions mapped to ArchiMate’s Infrastructure Service.

8.2 Enumerations

DAF 7.0 defines 38 enumerations. They type the tagged values listed above.

Enumeration	Values	Used by
ActorTypes	Active Business Partner, Business Partner, External Worker, Internal Worker, Not Set, Passive Business Partner, Worker	dActor.ActorType
ArchitectureType	AMD64, ARM64, RISC-V, ppc64le, s390x, x86	dDeploymentNode.ArchitectureType
CapabilityBusinessValue	Differentiator, High, Low, Medium, Support	dCapability.BusinessValue
dApplicationStrategy	Refactor, Rehost, Replatform, Retain, Retire, Rewrite	dApplicationComponent.InvestmentStrategy
dApplicationType	COTS, Core, Custom, DB, Instance, Module, Web Site	dApplicationComponent.ApplicationType
dAssumptionType	Application, Business, Data, Enterprise, System, Technology	dGoal.AssumptionType
dBaseStatus	Approved, Modified, Proposed, Rejected, Released, Solved, Validated	not used by a property
dboolean	false, true	dBusinessProcess.isAdopted, dBusinessProcess.isDocumented, dBusinessProcess.isEffective, dBusinessProcess.isOptimized, dDataEntity.isAccessible, dDataEntity.isTimely, dFeature.isImplemented, dIPRange.Public, dLogicalAppComponent.isMeetingBusinessNeeds, dLogicalAppComponent.isMeetingFutureNeeds, dLogicalAppComponent.isUsable, dRole.isSkillsAvailable, dRole.isSkillsDefined

Enumeration	Values	Used by
dCloudProvider	AWS, Azure, DigitalOcean, Google, Other	dPublicCluster.provider
dConstraintType	Invariant, Post-condition, Pre-condition, Process	not used by a property
dCostNature	Capex, Opex	dCostPeriod.costNature
dEventNature	Occurred, Possible, Recurring	dEvent.nature
dEventType	Business, External, Technical, Temporal	dEvent.eventType
dFeatureStatus	Implemented, Proposed, Rejected, Validated	not used by a property
dIncrement	Defined, Initial, Managed, Measured, None, UnderDevelopment	dCapability.Increments, dCapability.IncrementsToBe, dCapability.IncrementSupplyChain, dCapability.IncrementVertical
dLifecycleStatus	In Development, Live, Phasing Out, Proposed, Retired	not used by a property
dPrincipleType	Application Principle, Business Principle, Data Principle, Guiding Principle, Integration Principle, Technology Principle	dPrinciple.Type
dProcessType	Automatic, Client facing, Manual	dBusinessProcess.ProcessType

Enumeration	Values	Used by
dQuantityKind	Actual, Assumption, Estimate, unspecified	dQuantityValue.kind
dRequirementStatus	Mandatory, Proposed, Validated	not used by a property
dRequirementType	Business Rule, Performance, Persistence, Scalability, Security, Transport	dRequirement.Type
dRiskStrategy	Terminate, Tolerate, Transfer, Treat	dRisk.RiskStrategy
dSatisfactionType	Highly Dissatisfied, Highly Satisfied, Moderately Satisfied, Not Satisfied, Satisfied	not used by a property
dScala	High, Low, Medium	dStakeholder.Attitude, dStakeholder.Knowledge, dStakeholder.Legitimacy, dStakeholder.Power, dStakeholder.Urgency
executionMode	asynchronous, synchronous	dPhysicalService.execution_mode
format	<none>, date, date-time, email, hostname, idn-email, idn-hostname, ipv4, ipv6, iri, iri-reference, json-pointer, regex, relative-json-pointer, time, uri, uri-reference, uri-template	dJSON_SchemaSubSet.format, dJSON_Type.format
HTTPMethod	delete, get, head, options, patch, post, put, trace	dAPIOperation.hTTPMethod
InitiativeStatus	DeepGreen, Green, Orange, Red, Yellow	dInitiative.initiativeHealth

Enumeration	Values	Used by
ipRangeType	Pod, Service, subNetwork	dIPRange.ipRangeType
MimeTypes	application/json, application/octet-stream, application/pdf, application/problem+json, application/x-www-form-urlencoded, application/xml, application/yaml, multipart/form-data, text/csv, text/event-stream, text/html, text/plain	dAPI.consumes, dAPI.produces
NetworkQuality	High, Low, Medium	not used by a property
NetworkTypes	LAN, Mesh, WAN	dNetwork.NetworkType
ProcessCriticality	High, Low, Medium, unknown	dBusinessProcess.ProcessCriticality
ProtocolTypes	All, RNS, TCP, UDP, http, https	dPhysicalService.protocol
QualityNetwork	High, Low, Medium	not used by a property
SchemaVersion	http://json-schema.org/draft-04/schema#, http://json-schema.org/draft-05/schema#, http://json-schema.org/draft-06/schema#, http://json-schema.org/draft-07/schema#	dJSON_Schema.schema

Enumeration	Values	Used by
SchemeType	http, https, ws, wss	dAPI.schemes
ValueStreamCriticality	Limited Impact, Moderate Impact, Negligible Impact, Not Analyzed, Noticeable Impact, Significant Impact	dValueStream.Criticality

8.3 Model Checks

Model checks are implemented using Bellekens EA Validator. See Enterprise Architect Toolpack releases

The following 84 checks are implemented in Checks/checks. CIM, PIM and PSM follow the MDA levels; IDs are unique within a category.

Category / Type	Check (ID and description)	Rationale
All		
	001 An Element with the status not set	A model element must have the status set
	002 Elements still in status Proposed must be reviewed	Proposed status indicates that the Element is probably not managed
	003 The main DAF diagram without objects	No diagram should be empty.
	004 a Diagram must not contain more than 25 elements	Ideally a diagram should not contains more that seven elements
	005 a Diagram should not contain less than 2 elements	Ideally a diagram should contains at least Two elements
	006 Element is not displayed in a diagram	an element that is not deleted must be displayed in a least a diagram
CIM		
Application Architecture	001 A “LogicalAppComponent” should exposes a “BusinessService”	a 'dLogicalAppComponent ' element should be connected to a 'dBusinessService' element by a ' exposes Biz Service' Connector
Business Architecture	001 Goal is Operationalized By Capability	a 'dCapability' element should be connected to a ' dGoal' element by a 'isOperationalizedBy' Connector

Category / Type	Check (ID and description)	Rationale
	002 Capability is Measured By Measurement Indicator	a 'dCapability' element should be connected to a 'dMeasurementIndicator' element by a 'isMeasuredByf' Connector
	003 Logical Component Supports Capability	a 'dCapability' element should be connected to a 'dLogicalAppComponent' element by a 'Supports Capability' Connector
	004 Business Process Ensures the correct Operation of Capability	a 'dCapability' element should be connected to a 'dBusinessProcess' element by a 'Ensure the correct Operation' Connector
	006 Role executes Capability	a 'dCapability' element should be connected to a Role element
	007 Data Entity informs Capability	a 'dCapability' element should be connected to a Data Entity element
	008 Initiative Increases Maturity Of Capability	a 'dCapability' element can be connected to am Initiative element
	09 Biz Area without Capability	Am area aggregatyes business capabilities
	10 Resource without Capability	A logical application should support the business capability
Requirement Management	001 Goal related to Requirement	a 'dRequirement' element should be connected to a 'dGoal' element by a 'is related to' Connector
	002 Requirement isSatisfiedBy Feature	a 'dFeature' element should be connected to a ' dRequirement' element by a 'isSatisfiedBy' Connector
	003 Goal must relate to Requirement or another goal	a Goal element should be connected to a 'Goal' or a requirement
	004 Requirement is connected with another requirement OR a feature	a Goal element should be connected to a 'Goal' or a requirement
	005 Requirement cannot be connected Feature and Issue	a 'dFeature' element should be connected to a ' dRequirement' element by a 'isSatisfiedBy' Connector
Technology Architecture	001 Network Contains DeploymentNode	a 'dNetwork' element should be connected to a 'dDeploymentNode' element by a 'NetworkContainsNode' Connector
	002 Location Hosts DeploymentNode	a 'dLocation' element should be connected to a 'dDeploymentNode' element by a 'Hosts Node' Connector
PIM		
Application Architecture	001 Function has no documentation	Function has no documentation

Category / Type	Check (ID and description)	Rationale
	002 ApplicationComponent documentation contains long dash '–' character	dApplicationComponent documentation contains long dash '–' character
	003 Application Component does not have documentation	ApplicationComponent does not have documentation
	004 dApplicationComponent name contains 2 spaces between words	dApplicationComponent name contains 2 spaces between words
	005 dApplicationComponent name contains leading or trailing space characters	dApplicationComponent name contains leading or trailing space characters
	006 dApplicationComponent documentation is in HTML format (rather than plain text)	dApplicationComponentdocumentation is in HTML format (rather than plain text)
	007 Application components has no name	Application requires a name
	008 Function has no name	Function requires a name
	009 a function should be associated with an Application	an Application Function must be aggregated by an Application Component (dApplicationAggregatesFunction)
	010 ApplicationComponent Aggregates ApplicationFunction	a 'dApplicationComponent' element should be connected to a 'dApplicationFunction' element by a 'dApplicationAggregatesFunction' Connector
	013 Component Realizes Logical Component	a 'dApplicationComponent' element should be connected to a ' dLogicalAppComponent' element by a 'Realizes' Connector
	015 System configuring an Application Component	a 'dApplicationComponent' element should be configured by a 'dSystem' element by a 'configured by' Connector
	016 ApplicationComponent Exposes PhysicalService	a 'dApplicationComponent' element should be connected to a 'dPhysicalService' element by a 'Exposes Service' Connector
	017 PhysicalService Implements BusinessService	a 'dPhysicalService' element should be connected to a 'dBusinessService' element by a 'Implements' Connector
	021 Controller Implements Function	a 'dController' element should be connected to a 'dPhysicalService' element by a 'Implements Service' Connector
	022 Controller Controlls ModelClass	a 'dController' element should be connected to a 'dModelClass' element by a 'Controlls' Connector
	023 Controller should Implementa BusinessUseCase	a 'dController' element should be connected to a 'dBusinessUseCase' element by a 'isImplementedBy' Connector

Category / Type	Check (ID and description)	Rationale
	024 Controller Governs View	a 'dController' element should be connected to a 'dView' element by a 'Governs' Connector
Business Architecture	001 Business Use Case is is realizing Feature	a BusinessUseCase' element should be connected to a 'dFeature' element by a 'is realized by Use Case' Connector
	002 BusinessService isDescribedBy BusinessUseCase	a 'dBusinessUseCase' element should be connected to a 'dBusinessService' element by a 'isDescribedBy' Connector
	003 Use Cases that are not in exactly one Boundary	A UseCase must appear in exactly 1 Boundary (of all Use Case-diagrams)
Technology Architecture	001 ApplicationComponent isPhysically Hosted By Deployment Node	a 'dDeploymentNode' element should be connected to a 'dApplicationComponent' element by a 'isPhysicallyHostedBy' Connector
PSM		
Data Architecture	000 Class diagrams with more than 25 elements	A Class diagram should have max. 25 elements
	001 Unreferenced Enumerations	Search for unnecessary Enumerations
	002 Unreferenced Datatypes	zll the ata type must be described in a class
	003 Enumerations without values	An Enumeration must have at least 1 value.
	004 Model class has no name (or empty name)	Entity has no name (or empty name)
	005 Association class related to other class(es)	
	006 Attribute documentation is in HTML format	HTML format can generate issues
	007 Association documentation contains long dash '–' character	Association documentation contains long dash '–' character
	008 Association documentation is in HTML format (rather than plain text)	Association documentation is in HTML format (rather than plain text)
	009 Attribute documentation contains long dash '–' character	Add documentation
	010 Datatype/Enumeration documentation contains long dash '–' character	Datatype/Enumeration documentation contains long dash '–' character
	011 Datatype/Enumeration documentation is in HTML format (rather than plain text)	Datatype/Enumeration documentation is in HTML format (rather than plain text)

Category / Type	Check (ID and description)	Rationale
	012 Datatype/Enumeration does not have documentation	Datatype/Enumeration does not have documentation
	013 Classes containing Attributes without datatype	Every Attribute of a Class must have a Datatype
	014 Two classes with the same name in the model	classes with the same name can be confusing
	015 Mode Class without any relations to other entities	Entities without any relations to other entities. This validation skips entities which are marked as >
	016 Model Class does not have Documentation	Entity does not have Documentation
	017 Model Class is directly derived from more than one base class	Entity is directly derived from more than one base class
	018 Entities (classes or association classes) which do not appear in any diagram in the model	Entities (classes or association classes) which do not appear in any diagram in the model
	019 Attribute is not defined as public	ensure that the attribute need to be != public
	020 Attribute has no name (or empty name)	Attribute must have a name
	021 Attribute of an entity does not have a type	Attribute of an entity does not have a type
	022 Unnamed association (between 2 model Classes)	all associations must have a name
	023 Unnamed self-association	Unnamed self-association
	024 Association name contains space characters	Association name cannot contains spaces
	025 Duplicate associations (association with exactly the same name)	associations must have unique names
	026 Attribute has no documentation	All Attribute should have a Description.
	027 association should specify navigability	unspecified navigability not recommended for information model
Implementation	001 Initiative IncreasesMaturityOf Capability	a 'dInitiative' element should be connected to a 'dCapability' element by a 'IncreasesMaturityOf' Connector
	002 Decision are attached to Initiative	a Decision element should be connected to a Initiative element by a 'IncreasesMaturityOf' Connector

Category / Type	Check (ID and description)	Rationale
Technology	001 API does not have base path	Base path is required
	002 API does not have Documentation	Explain the purpose of the API
	003 Operation is not defined as public	operations that are set as Package or Private have special meaning
	004 APIoperation has no documentation	non documented Operations are confusing
	005 API Operation with no TAG	Tag is necessary
	005 API Operation with no relative path	relative path is necessary
	010 API does not have contact	contact is required
	011 API does not have license	license is required
	012 API does not have host	host is required

8.4 DAF COLORS

The metamodel uses certain colors to indicate domain of architecture

8.4.1 RGB Colors

- DAF Generic blue: 107,203,236
- Enterprise ARCHITECTURE Green: 155,187,89
- SOLUTION ARCHITECTURE dark blue: 79,129,189
- DESIGN ARCHITECTURE **Dark Orange: 233,191,149**
- Performance **violet: 234,198,234**
- Requirements **red: 222,103,103**
- Stakeholder **light blue: 240,255,255**

- Technology **Green: 156,204,84**
- Strategy **Orange: 247,150,70**
- COMMON **yellow: 241,224,151**

8.4.2 HEX Colors

- Blue: 107,203,236 → #6BCBEC
- - Green: 155,187,89 → #9BBB59
- - Dark Blue: 79,129,189 → #4F81BD
- - Orange: 247,150,70 → #F79646
- - Dark Orange: 233,191,149 → #E9BF95
- - Violet: 234,198,234 → #EAC6EA
- - Red: 222,103,103 → #DE6767
- - Light Blue: 240,255,255 → #F0FFFF
- - Green (another shade): 156,204,84 → #9CCC54
- - Yellow: 241,224,151 → #F1E097